MvCodeReader SDK (C or C++)

Developer Guide

Legal Information

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE DOCUMENT IS PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". OUR COMPANY MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT. IN NO EVENT WILL OUR COMPANY BE LIABLE FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, IN CONNECTION WITH THE USE OF THE DOCUMENT, EVEN IF OUR COMPANY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.

Contents

Chapter 1 Overview

The MvCodeReaderSDK is a software development kit based on Machine Vision Camera SDK, which can cooperate with X86 smart code readers, VPU (Video Processing Unit) vision sensors, and other code readers to acquire image with code information, set parameters, and output information.

1.1 Introduction

This manual mainly introduces the MvCodeReaderSDK based on C/C++ language, which provides several external API for controlling code reader and getting code information.

Note

Some APIs are supported by the virtual code reader, related details are in the corresponding API definition. The virtual code reader can be connected or configured via the tool VirtualCamTool.exe, you can refer to the IDMVS Client user manual for detailed virtual code reader parameters.

1.2 Development Environment

The development environment of MvCodeReader is shown in the table below.

Item	Required
CPU	Intel Pentium IV 3.0 GHZ and above
RAM	4 GB and above
Network Interface Card	Gigabit NIC with jumbo frame enabled
Operating System	Microsoft® Windows XP (32-bit)/Windows 7 (32/64-bit)/Windows 10 (32/64-bit)

1.3 Update History

The update history shows the summary of changes in MvCodeReaderSDK with different versions.

Summary of Changes in Version 1.5.3_01/2024

Version	Content
Version 1.5.3_Jan/2024	1. Added the function of preventing illegal occupation for camera.
	2. Added an API for setting the number of frame data cache nodes: ***MV_CODEREADER_SetFrameNodeNum*** .
	3. Added an API for getting the timeout of GVCP commands: ***MV_CODEREADER_GIGE_GetGvcpTimeout*** .
	4. Added an API for clearing the cache of the GenICam node: ***MV_CODEREADER_InvalidateNodes*** .
	5. Extended the structure about device information ***MV_CODEREADER_DEVICE_INFO*** : added one member **nDeviceType** (device type).
	6. Extended the structure about output frame information which contains barcode information ***MV_CODEREADER_IMAGE_OUT_INFO*** , the structure about output frame information which contains bar code and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** , and the structure about output frame information which contains bar code (with QR code) and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** : added one member **nWholeFlag** (marker of complete image).
	7. Extended the enumeration about bar code type ***MV_CODEREADER_CODE_TYPE*** : added one member **MV_CODEREADER_TDCR_HANXIN** (Hanxin code).
	8. Added the SDK generating Dump function.

Summary of Changes in Version 1.5.2_09/2023

Version	Content
Version 1.5.2_Sep/2023	1. Added the waybill rotation function.
	2. Extended the structure about bar code information including quality information ***MV_CODEREADER_BCR_INFO_EX*** and structure about extended bar code information including quality information ***MV_CODEREADER_BCR_INFO_EX2*** : added 4 members:

Version	Content
	nTriggerTimeTvHigh (high-order 32-bit of trigger start time, unit: second), **nTriggerTimeTvLow** (low-order 32-bit of trigger start time, unit: second), **nTriggerTimeUtvHigh** (high-order 32-bit of trigger start time, unit: microsecond), **nTriggerTimeUtvLow** (low-order 32-bit of trigger start time, unit: microsecond).
	3. Added enumeration about trigger source: ***MV_CODEREADER_TRIGGER_SOURCE*** .
	4. Added 4 enumeration types **MV_CODEREADER_TDCR_MICROQR**, **MV_CODEREADER_BCR_PHARMACODE**, **MV_CODEREADER_BCR_PHARMACODE2D**, and **MV_CODEREADER_TDCR_AZTEC**: ***MV_CODEREADER_CODE_TYPE*** .
	5. Added 5 error codes: MV_CODEREADER_E_UPG_PLATFORM_DISMATCH, MV_CODEREADER_E_UPG_TYPE_DISMATCH, MV_CODEREADER_E_UPG_SPACE_DISMATCH, MV_CODEREADER_E_UPG_MEM_DISMATCH, MV_CODEREADER_E_UPG_NET_TRANS_ERROR. See details in ***Error Code*** .

Summary of Changes in Version 1.5.1_06/2023

Version	Content
Version 1.5.1_June/2023	1. Fixed the problem of memory leak under high bandwidth usage.
	2. Optimized memory usage of the SDK.
	3. Optimized the size of the SDK package.

Summary of Changes in Version 1.5.0_09/2022

Version	Content
Version 1.5.0_Sept./2022	1. Added an API for enumerating devices that are connected via the private protocol: ***MV_CODEREADER_EnumIDDevices*** .
	2. Added an API for setting the timeout of GVCP commands: ***MV_CODEREADER_GIGE_SetGvcpTimeout*** .
	3. Added an API for registering the callback function for trigger information: ***MV_CODEREADER_RegisterTriggerCallBack*** .
	4. Extended the structure about bar code information including quality information ***MV_CODEREADER_BCR_INFO_EX*** and structure about extended bar code information including quality information ***MV_CODEREADER_BCR_INFO_EX2*** : added one member **nTotalProcCost** (time duration from trigger start time to app output time).
	5. Added the following error codes: MV_CODEREADER_E_UPG_BLE_DISCONNECT, MV_CODEREADER_E_UPG_BATTERY_NOTENOUGH, MV_CODEREADER_E_UPG_RTC_NOT_PRESENT, MV_CODEREADER_E_UPG_APP_ERR, MV_CODEREADER_E_UPG_L3_ERR, and MV_CODEREADER_E_UPG_MCU_ERR. See details in ***Error Code*** .

Summary of Changes in Version 1.4.0_02/2022

Version	Content
Version 1.4.0_Feb./2022	1. Extended the structure about waybill information ***MV_CODEREADER_WAYBILL_INFO*** : added one member **nOcrRowNum** (the number of OCR rows of current waybill) with one reserved byte.
	2. Extended the structure about waybill information list ***MV_CODEREADER_WAYBILL_LIST*** : added one member **nOcrAllNum** (the total number of OCR rows of all waybills) with one reserved byte.
	3. Added one structure about OCR basic information: ***MV_CODEREADER_OCR_ROW_INFO*** .
	4. Added one structure about OCR information list: ***MV_CODEREADER_OCR_INFO_LIST*** .

Version	Content
	5. Extended the structure about output frame information which contains bar code and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** : added one member **UnparsedOcrList** (unparsed OCR information list).
	6. Extended the structure about output frame information which contains bar code and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** : added one member **UnparsedOcrList** (unparsed OCR information list).

Summary of Changes in Version 1.3.0_07/2021

Version	Content
Version 1.3.0_July/2021	1. Extended the structure about bar code information including quality information ***MV_CODEREADER_BCR_INFO_EX*** : added one member **n1DIsGetQuality** (whether the device supports 1D code quality grading).
	2. Extended the structure about bar code quality information ***MV_CODEREADER_CODE_INFO*** : added members for the scoring and grading of 1D codes.
	3. Extended the structure about GigE device information ***MV_CODEREADER_GIGE_DEVICE_INFO*** : added one member **nCurUserIP** (the IP of the user occupying the current device, this member is valid when the RunTime version of MVCameraSDK is 3.5.1 or later).
	4. Added one structure about extended bar code information list: ***MV_CODEREADER_RESULT_BCR_EX2*** . Currently, the maximum supported length of a bar code is 1,023 bytes. For ***MV_CODEREADER_RESULT_BCR*** and ***MV_CODEREADER_RESULT_BCR_EX*** , if the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.

Version	Content
	5. Extended the structure about output frame information which contains bar code and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** : added one parameter **pstCodeListEx2** (the pointer to the extended bar code information list ***MV_CODEREADER_RESULT_BCR_EX2***). It is recommended to use the new bar code structure to parse bar code information.
	6. Updated the sample codes. See details in ***Sample Codes*** .

Summary of Changes in Version 1.2.0_04/2021

Version	Content
Version 1.2.0_April/2021	1. Parts of APIs are supported by the virtual code reader, which can be connected or configured via the tool VirtualCamTool.exe.
	2. Provided the Runtime installation package, which supports a silent installation.
	3. Updated the matting algorithm library to Version 1.7.2.
	4. Extended the structure about bar code information including quality information ***MV_CODEREADER_BCR_INFO_EX*** : added one member **nIDRScore** (code score) with one reserved byte.
	5. Extended the enumeration about bar code types ***MV_CODEREADER_CODE_TYPE*** : added seven types of codes: **MV_CODEREADER_BCR_MATRIX25** (MATRIX25 code), **MV_CODEREADER_BCR_ITF14** (ITF-14 code), **MV_CODEREADER_BCR_MSI** (MSI code), **MV_CODEREADER_BCR_CODE11** (Code 11), **MV_CODEREADER_BCR_INDUSTRIAL25** (INDUSTRIAL25 code), **MV_CODEREADER_BCR_CHINAPOST** (CHINAPOST code), and **MV_CODEREADER_TDCR_ECC140** (ECC140 code).
	6. Extended the structure about bar code quality information ***MV_CODEREADER_CODE_INFO*** : added two members **nRMGrade** (reflectance margin quality level) and **fRMScore** (reflectance margin score) with two reserved bytes.
	7. Extended the structure about output frame information which contains bar code information

Version	Content
	MV_CODEREADER_IMAGE_OUT_INFO , structure about output frame information which contains bar code and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** , and structure about output frame information which contains bar code and waybill information ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** : added one member **nImageCost** (the algorithm processing time cost for recognizing the information contained in the image) with one reserved byte.
	8. Added one error code: MV_CODEREADER_E_FILE_PATH (incorrect file path).

Summary of Changes in Version 1.1.0_07/2020

Version	Content
Version 1.1.0_July/2020	1. Added one API for getting one image frame by using timeout mechanism (the obtained image information contains bar code quality information): ***MV_CODEREADER_GetOneFrameTimeoutEx2*** .
	2. Added one API for registering the callback function for image data, which contains bar code quality information: ***MV_CODEREADER_RegisterImageCallBackEx2*** .
	3. Added one API for getting one image frame of a specified channel by using timeout mechanism (the obtained image information contains bar code quality information): ***MV_CODEREADER_MSC_GetOneFrameTimeout*** .
	4. Added one API for registering the callback function for image data of a specified channel (the image data contains bar code quality information): ***MV_CODEREADER_MSC_RegisterImageCallBack*** .
	5. Added one API for getting the waybill information of input image: ***MV_CODEREADER_GetWayBillInfo*** .
	6. Added one API for getting the waybill information of input image (the obtained image information contains bar code quality): ***MV_CODEREADER_GetWayBillInfoEx*** .
	7. Added one API for registering the callback function for all events: ***MV_CODEREADER_RegisterAllEventCallBack*** .

Version	Content
	8. Extended the structure about bar code information ***MV_CODEREADER_BCR_INFO*** : added three members: **sPPM** (PPM), **sAlgoCost** (algorithm time cost), and **sSharpness** (image definition).
	9. Added one structure about output frame information which contains information of bar code, waybill, and bar code quality: ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** .
	10. Added one structure about bar code quality information: ***MV_CODEREADER_CODE_INFO*** .
	11. Added one structure about bar code information including quality information: ***MV_CODEREADER_BCR_INFO_EX*** .
	12. Extended the structures about output frame information ***MV_CODEREADER_IMAGE_OUT_INFO*** , ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** , and ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** : added one member **nChannelID** (Channel ID corresponding to the stream).
	13. Extended the enumeration about bar code type ***MV_CODEREADER_CODE_TYPE*** : added one bar code type **MV_CODEREADER_TDCR_PDF417** (PDF417 code).
	14. Extended the structure about bar code information including quality information: ***MV_CODEREADER_BCR_INFO_EX*** added one member **bIsGetQuality** (whether the device supports bar code scoring).

Summary of Changes in Version 1.0.0_04/2019

New document.

Chapter 2 Control Code Reader and Get Code Information

The code reader is a camera for reading 1D and 2D barcodes, and it consists of machine vision camera and industrial computer with some special algorithms. Here, you can follow the API calling flow to log in to a code reader, set the parameters, and acquire the images with code information.

Before You Start

- Make sure that the giant frame of network port is enabled.
- Make sure that the RunTime of MVS SDK has been installed properly, and the version should be 3.0.0 or higher.

Steps

Note

- The APIs for registering callback function of image data and APIs for getting image are mutually exclusive.
- You can call API ***MV_CODEREADER_MSC_RegisterImageCallBack*** to register a callback function for image data of a specified channel, and you can call this API repeatedly to register callback functions for image data and bar code information of multiple channels.
- You can call API ***MV_CODEREADER_MSC_GetOneFrameTimeout*** to get one picture frame of a specified channel, and you can poll this API repeatedly to get picture frames and bar code information of multiple channels.

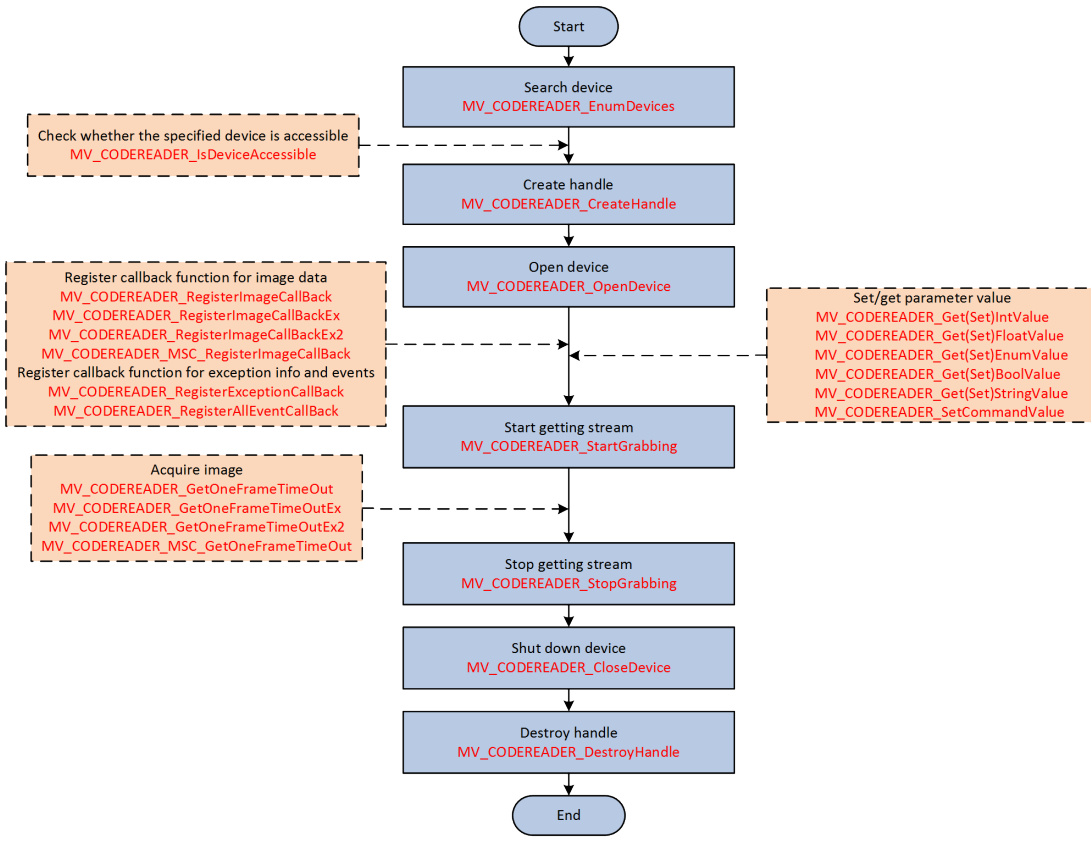

Figure 2-1 Programming Flow of Controlling Code Reader and Getting Code Information

1. Call ***MV_CODEREADER_EnumDevices*** to search device.
2. **Optional:** Call ***MV_CODEREADER_IsDeviceAccessible*** to check whether the specified device is accessible.
3. Call ***MV_CODEREADER_CreateHandle*** to create the handle.
4. Call ***MV_CODEREADER_OpenDevice*** to open the device.
5. **Optional:** Get the default or configured parameter value for reference.
   - Call ***MV_CODEREADER_GetIntValue*** to get parameter values of integer type.
   - Call ***MV_CODEREADER_GetFloatValue*** to get parameter values of float type.
   - Call ***MV_CODEREADER_GetEnumValue*** to get parameter values of enumerated type.
   - Call ***MV_CODEREADER_GetBoolValue*** to get parameter values of boolean type.
   - Call ***MV_CODEREADER_GetStringValue*** to get parameter values of string type.
6. **Optional:** Set the parameter value.
   - Call ***MV_CODEREADER_SetIntValue*** to set parameter values of integer type.
   - Call ***MV_CODEREADER_SetFloatValue*** to set parameter values of float type.
   - Call ***MV_CODEREADER_SetEnumValue*** to set parameter values of enumerated type.
   - Call ***MV_CODEREADER_SetBoolValue*** to set parameter values of boolean type.
   - Call ***MV_CODEREADER_SetStringValue*** to set parameter values of string type.
   - Call ***MV_CODEREADER_SetCommandValue*** to set parameter values of command type.
7. Register a callback function for image data or exception information.

- Call ***MV_CODEREADER_RegisterImageCallBack*** to register the callback function for image data.
- Call ***MV_CODEREADER_RegisterImageCallBackEx*** to register the callback function for image data.
- Call ***MV_CODEREADER_RegisterImageCallBackEx2*** to register the callback function for image data.
- Call ***MV_CODEREADER_MSC_RegisterImageCallBack*** to register the callback function for image data of a specified channel.
- Call ***MV_CODEREADER_RegisterExceptionCallBack*** to register the callback function for device exception information.
- Call ***MV_CODEREADER_RegisterAllEventCallBack*** to register the callback function for all events.

8. Call ***MV_CODEREADER_StartGrabbing*** to start streaming.
9. **Optional:** Acquire images with code information.
   - Call ***MV_CODEREADER_GetOneFrameTimeout*** to acquire images with code information.
   - Call ***MV_CODEREADER_GetOneFrameTimeoutEx*** to acquire images with code and waybill information.
   - Call ***MV_CODEREADER_GetOneFrameTimeoutEx2*** to acquire images with code and waybill information.
   - Call ***MV_CODEREADER_MSC_GetOneFrameTimeout*** to acquire images with code and waybill information of a specified channel.
10. Call ***MV_CODEREADER_StopGrabbing*** to stop streaming.
11. Call ***MV_CODEREADER_CloseDevice*** to close the device.
12. Call ***MV_CODEREADER_DestroyHandle*** to destroy the handle.

Chapter 3 API Reference

3.1 General APIs

This section introduces APIs of getting the SDK version information, initializing, getting the device information, and other basic operations.

3.1.1 MV_CODEREADER_GetSDKVersion

Get the SDK version number.

API Definition

```
unsigned int MV_CODEREADER_GetSDKVersion( );
```

Return Value

Return SDK version number for success, for example, 0×01000000 indicates V1.0.0.0, and the format is as follows:

Main	Sub	Revision	Test
8bits	8bits	8bits	8bits

3.1.2 MV_CODEREADER_EnumDevices

Enumerate devices. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_EnumDevices(
  MV_CODEREADER_DEVICE_INFO_LIST   *pstDevList,
  unsigned int                     nTLayerType
);
```

Parameters

pstDevList

[IN][OUT] Device list. See ***MV_CODEREADER_DEVICE_INFO_LIST*** for details.

nTLayerType

[IN] Transport layer type. The default type is GigE. Refer to the device type in ***Macro Definition*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Enumerating Devices

```
nRet = MV_CODEREADER_EnumDevices(&stDeviceList, MV_CODEREADER_GIGE_DEVICE |
MV_CODEREADER_USB_DEVICE);
 if (MV_CODEREADER_OK != nRet)
 {
    printf("Enum Devices fail! nRet [0x%x]\n", nRet);
    break;
 }
```

3.1.3 MV_CODEREADER_EnumCodeReader

Enumerate specified devices. This API supports enumerating virtual code readers, but specifying device series is not supported.

API Definition

```
int MV_CODEREADER_EnumCodeReader(
  MV_CODEREADER_DEVICE_INFO_LIST    *pstDevList
);
```

Parameters

pstDevList

[IN][OUT] Device list. See ***MV_CODEREADER_DEVICE_INFO_LIST*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.4 MV_CODEREADER_EnumIDDevices

Enumerate devices that are connected via the private protocol. This API is not supported by virtual code readers.

API Definition

```
int MV_CODEREADER_EnumIDDevices(
  MV_CODEREADER_DEVICE_INFO_LIST    *pstDevList
);
```

Parameters

pstDevList

[IN][OUT] Device list. See ***MV_CODEREADER_DEVICE_INFO_LIST*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.5 MV_CODEREADER_IsDeviceAccessible

Check if the specified device is accessible. This API cannot check if the virtual code reader is accessible.

API Definition

```
bool MV_CODEREADER_IsDeviceAccessible(
  MV_CODEREADER_DEVICE_INFO          *pstDevInfo,
  unsigned int                       nAccessMode
);
```

Parameters

pstDevInfo

[IN] Device information, see ***MV_CODEREADER_DEVICE_INFO*** for details.

nAccessMode

[IN] Access permission, see the table "Device Access Mode" of ***Macro Definition*** for details.

Return Value

Whether the device is accessible.

3.1.6 MV_CODEREADER_CreateHandle

Create device handle. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_CreateHandle(
  void                          **handle,
  const MV_CODEREADER_DEVICE_INFO   *pstDevInfo
);
```

Parameters

handle

[IN] [OUT] Handle.

pstDevInfo

[IN] Device information, see ***MV_CODEREADER_DEVICE_INFO*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Creating Device Handle

```
nRet = MV_CODEREADER_CreateHandle(&handle, stDeviceList.pDeviceInfo[nIndex]);
 if (MV_CODEREADER_OK != nRet)
 {
    printf("Create Handle fail! nRet [0x%x]\n", nRet);
    break;
 }
```

3.1.7 MV_CODEREADER_CreateHandleBySerialNumber

Create device handle by serial number. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_CreateHandleBySerialNumber(
  void        **handle,
  const char   *chSerialNumber
);
```

Parameters

handle

[IN] [OUT] Handle.

chSerialNumber

[IN] Device serial number.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.8 MV_CODEREADER_DestroyHandle

Destroy handle. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_DestroyHandle(
  void    *handle
);
```

Parameters

handle

[IN] Handle.

Remarks

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Destroying Handle

```
nRet = MV_CODEREADER_DestroyHandle(m_handle);
if (MV_CODEREADER_OK != nRet)
{
   cstrInfo.Format(_T("Destroy handle failed! err code:%#x"), nRet);
   MessageBox(cstrInfo);
   return ;
}
```

3.1.9 MV_CODEREADER_OpenDevice

Open device. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_OpenDevice(
  void                *handle
);
```

Parameters

handle

[IN] Handle.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Opening Device

```
nRet = MV_CODEREADER_OpenDevice(handle);
if (MV_CODEREADER_OK != nRet)
```

```
{
  printf("Open Device fail! nRet [0x%x]\n", nRet);
  break;
}
```

3.1.10 MV_CODEREADER_CloseDevice

Shut down device. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_CloseDevice(
  void                *handle
);
```

Parameters

handle

[IN] Handle.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Shutting Down Device

```
MV_CODEREADER_CloseDevice(m_hDevHandle);
nRet = MV_CODEREADER_DestroyHandle(m_hDevHandle);
m_hDevHandle = NULL;
return nRet;
```

3.1.11 MV_CODEREADER_StartGrabbing

Start streaming. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_StartGrabbing(
  void                *handle
);
```

Parameters

handle

[IN] Handle.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.12 MV_CODEREADER_StopGrabbing

Stop streaming. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_StopGrabbing(
  void               *handle
);
```

Parameters

handle

[IN] Handle.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.13 MV_CODEREADER_GetDeviceInfo

Get device information. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GetDeviceInfo(
  void                    *handle,
  MV_CODEREADER_DEVICE_INFO      *pstDevInfo
);
```

Parameters

handle

[IN] Handle.

pstDevInfo

[IN] [OUT] Device information, see ***MV_CODEREADER_DEVICE_INFO*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.2 Functional APIs

This section introduces APIs of saving images, forcing setting the IP address, configuring the IP address, reading files, getting files, and other functions.

Note

The functional APIs are not supported by the virtual code reader.

3.2.1 MV_CODEREADER_GIGE_ForceIp

Force setting IP. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GIGE_ForceIp(
  void           *handle,
  unsigned int    nIP,
  unsigned int    nSubNetMask,
  unsigned int    nDefaultGateWay
);
```

Parameters

handle

[IN] Handle.

nIP

[IN] Configured IP.

nSubNetMask

[IN] Subnet mask

nDefaultGateWay

[IN] Default gateway

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

The API is to be compatible with the SI-series smart code reader. It only supports error reporting for the IP configuration, while not support error reporting for the subnet mask and gateway configurations. When you set the **nIP** to 0, the device IP mode is set to DHCP.

Example

Sample Code of Force Setting IP

```
nRet = MV_CODEREADER_GIGE_ForceIp(m_handle, m_dwIpaddress, m_dwSubNetMask, m_dwDefaultGateWay);
if (MV_CODEREADER_OK != nRet)
{
   ShowErrorMsg(TEXT("Set forceIp fail"), nRet);
   throw nRet;
}
```

3.2.2 MV_CODEREADER_GIGE_SetGvcpTimeout

Set the timeout of GVCP commands. This API is invalid for virtual code readers. For virtual code readers, success (MV_CODEREADER_OK) is returned, but the configuration will not take effect.

API Definition

```
int MV_CODEREADER_GIGE_SetGvcpTimeout(
  void              *handle,
  unsigned int      nMillisec
);
```

Parameters

handle

[IN] Device handle.

nMillisec

[IN] Timeout, unit: millisecond, value range: [0, 10000].

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

After the device is connected, you can call this API to set the timeout of GVCP commands.

3.2.3 MV_CODEREADER_GIGE_GetGvcpTimeout

Get the timeout of GVCP commands. This API is invalid for virtual code readers. For virtual code readers, success (MV_CODEREADER_OK) is returned, but the configuration will not take effect.

API Definition

```
int MV_CODEREADER_GIGE_GetGvcpTimeout(
  void              *handle,
  unsigned int      *pMillisec
);
```

Parameters

handle

[IN] Device handle.

pMillisec

[IN][OUT] Timeout pointer, unit: millisecond, value range: [0, 10000].

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

After the device is connected, you can call this API to get the timeout of GVCP commands.

3.2.4 MV_CODEREADER_GIGE_SetIpConfig

Set device IP configuration type. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GIGE_SetIpConfig(
  void              *handle,
  unsigned int      nType
);
```

Parameters

handle

[IN] Handle.

nType

[IN] IP configuration type, see the ***IP Configuration Mode*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

The API is not supported by SI-series smart code reader for configuring the IP configuration type. If you set the IP mode of SI-series smart code reader to STATIC, OK will be returned. If you set the IP mode to LLA, the error "The function is not supported" will be returned. To set the IP mode to DHCP, you should set the parameter **nIP** of API ***MV_CODEREADER_GIGE_ForceIp*** to 0.

3.2.5 MV_CODEREADER_FileAccessRead

Read files from camera. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_FileAccessRead(
  void                    *handle,
  MV_CODEREADER_FILE_ACCESS    *pstFileAccess
);
```

Parameters

handle

[IN] Handle.

pstFileAccess

[IN] File access structure, see ***MV_CODEREADER_FILE_ACCESS*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.2.6 MV_CODEREADER_GetFileAccessProgress

Get file access progress. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GetFileAccessProgress(
  void                          *handle,
  MV_CODEREADER_FILE_ACCESS_PROGRESS      *pstFileAccessProgress
);
```

Parameters

handle

[IN] Handle.

pstFileAccessProgress

[IN][OUT] File access progress, see ***MV_CODEREADER_FILE_ACCESS_PROGRESS*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.2.7 MV_CODEREADER_FileAccessWrite

Write files to the camera. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_FileAccessWrite(
  void                    *handle,
  MV_CODEREADER_FILE_ACCESS    *pstFileAccess
);
```

Parameters

handle

[IN] Handle.

pstFileAccess

[IN] File access structure, see ***MV_CODEREADER_FILE_ACCESS*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.2.8 MV_CODEREADER_SaveImage

Save image. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_SaveImage(
  void                          *handle,
  MV_CODEREADER_SAVE_IMAGE_PARAM_EX     *pSaveParam
);
```

Parameters

handle

[IN] Handle

pSaveParam

[IN][OUT] Image parameters, see ***MV_CODEREADER_SAVE_IMAGE_PARAM_EX*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

This API is used for image type conversion, supports convert image storage format RAW to JEPG or BMP. The supported JPEG quality range is: (50-99]. You should save the image to your local PC.

3.2.9 MV_CODEREADER_SetFrameNodeNum

Set the number of frame data cache nodes. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_SetFrameNodeNum(
  void              *handle,
  unsigned int          nNodeNum = 3
);
```

Parameters

handle

[IN] Device handle.

nNodeNum

[IN] Number cache nodes: [1, 12], 3 by default.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

Call this API after device handle is created and before device is connected.

3.2.10 MV_CODEREADER_InvalidateNodes

Clear the cache of the GenICam node.

API Definition

```
int MV_CODEREADER_InvalidateNodes(
  void              *handle
);
```

Parameters

handle

[IN] Device handle.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.3 Image Acquisition APIs

This section introduces APIs of getting the image data.

Note

The image acquisition APIs are supported by the virtual code reader.

3.3.1 MV_CODEREADER_GetOneFrameTimeout

Get one picture frame by using timeout mechanism. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GetOneFrameTimeout(
  void                          *handle,
  unsigned char                 **pData,
  MV_CODEREADER_IMAGE_OUT_INFO  *pstFrameInfo,
  unsigned int                  nMsec
);
```

Parameters

handle

[IN] Handle.

pData

[IN] [OUT] Image data.

pstFrameInfo

[IN] [OUT] Image information, see ***MV_CODEREADER_IMAGE_OUT_INFO*** for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired when timeout period expired.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.3.2 MV_CODEREADER_GetOneFrameTimeoutEx

Get one image frame by using timeout mechanism (the obtained image information contains bar code and waybill information). This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GetOneFrameTimeoutEx(
  void                          *handle,
  unsigned char                 **pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX      *pstFrameInfo,
  unsigned int                  nMsec
);
```

Parameters

handle

[IN] Handle.

pData

[IN][OUT] Pointer to image data.

pstFrameInfo

[IN][OUT] Image information structure, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired when timeout period expired.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.3.3 MV_CODEREADER_GetOneFrameTimeoutEx2

Get one image frame by using timeout mechanism (the obtained image information contains information of bar code, waybill, and bar code quality). This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_GetOneFrameTimeoutEx2(
  void                                  *handle,
  unsigned char                         **pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX2      *pstFrameInfo,
  unsigned int                          nMsec
);
```

Parameters

handle

[IN] Handle.

pData

[IN][OUT] Pointer to image data.

pstFrameInfo

[IN][OUT] Image information structure, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired within the timeout period.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- This API supports getting the bar code quality information of ID5000-series smart code reader, while for the code readers without bar code scoring function, the information of bar code quality is 0.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.3.4 MV_CODEREADER_MSC_GetOneFrameTimeout

Get one picture frame of a specified channel by using timeout mechanism. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_MSC_GetOneFrameTimeout(
  void                          *handle,
  unsigned char                 **pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX2     *pstFrameInfo,
  unsigned int                  nChannelID,
  unsigned int                  nMsec
);
```

Parameters

handle

[IN] Handle.

pData

[IN][OUT] Pointer to image data.

pstFrameInfo

[IN][OUT] Image information (including bar code quality information), see ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** for details.

nChannelID

[IN] Channel ID. For single-channel firmwares, the channel ID is 0; for multi-channel firmwares, the channel ID starts from 0, and the number of channels depends on the number of sensors. For example, if the number of sensors is 3, the channel ID range is: [0,1,2].

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired within the timeout period.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- You can get the image information of different channels for IDX-series smart code reader by polling this API.
- This API supports getting image information of singe-channel firmwares and the input parameter **nChannelID** should be set to 0.
- This API supports getting the bar code quality information of ID5000-series smart code reader. While for the code readers without bar code scoring function, the information of bar code quality is 0.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.4 Data Callback APIs

This section introduces APIs of registering the callback function for exceptions, events, and the image data.

Note

Only APIs of registering the callback function for image data are supported by the virtual code reader.

3.4.1 MV_CODEREADER_RegisterAllEventCallBack

Register the callback function for all events. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_RegisterAllEventCallBack(
  void          *handle,
  void          (__stdcall* cbAllEvent)(MV_CODEREADER_EVENT_OUT_INFO* pEventInfo, void* pUser),
  void          *pUser
);
```

Parameters

handle

[IN] Handle.

cbAllEvent

[IN][OUT] Events callback function.

```
void (__stdcall* cbAllEvent)(
  MV_CODEREADER_EVENT_OUT_INFO   *pEventInfo,
  void                           *pUser
)
```

pEventInfo

[IN][OUT] All events information, see ***MV_CODEREADER_EVENT_OUT_INFO*** for details.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

This API is available only when the device supports nodes **ArithmeticModel** and **BillCombineEnable**. After calling this API, you need to configure these two nodes before this API takes effect.

3.4.2 MV_CODEREADER_RegisterExceptionCallBack

Get device exception message. This API is invalid to the virtual code reader.

API Definition

```
int MV_CODEREADER_RegisterExceptionCallBack(
  void          *handle,
  void          (__stdcall* cbException)(unsigned int nMsgType, void* pUser),
```

```
  void          *pUser
);
```

Parameters

handle

[IN] Handle.

cbException

[IN][OUT] Callback function to receive exception messages.

```
void (__stdcall* cbException)(
  unsigned int       nMsgType,
  void               *pUser
)
```

nMsgType

[OUT] Exception message type.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.4.3 MV_CODEREADER_RegisterImageCallBack

Register callback function for image data. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_RegisterImageCallBack(
  void          *handle,
  void          (__stdcall* cbOutput)(unsigned char * pData, MV_CODEREADER_IMAGE_OUT_INFO* pstFrameInfo,
void* pUser),
  void          *pUser
);
```

Parameters

handle

[IN] Handle.

cbOutput

[IN][OUT] Image data callback function.

```
void (__stdcall* cbOutput)(
  unsigned char               *pData,
  MV_CODEREADER_IMAGE_OUT_INFO   *pstFrameInfo,
  void                        *pUser
)
```

pData

[IN][OUT] Image data.

pstFrameInfo

[IN][OUT] Obtained frame information, see ***MV_CODEREADER_IMAGE_OUT_INFO*** for details.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.4.4 MV_CODEREADER_RegisterImageCallBackEx

Register the callback function for image data. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_RegisterImageCallBackEx(
  void       *handle,
  void       (__stdcall* cbOutput)(unsigned char *pData, MV_CODEREADER_IMAGE_OUT_INFO_EX* pstFrameInfo,
void* pUser),
  void       *pUser
);
```

Parameters

handle

[IN] Handle.

cbOutput

[IN][OUT] Image data callback function.

```
void (__stdcall* cbOutput)(
  unsigned char              *pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX    *pstFrameInfo,
  void                       *pUser
)
```

pData

[IN][OUT] Image data.

pstFrameInfo

[IN][OUT] Obtained frame information, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** for details.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.4.5 MV_CODEREADER_RegisterImageCallBackEx2

Register the callback function for image data. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_RegisterImageCallBackEx2(
  void        *handle,
  void        (__stdcall* cbOutput)(unsigned char * pData, MV_CODEREADER_IMAGE_OUT_INFO_EX2* pstFrameInfo,
void* pUser),
  void        *pUser
);
```

Parameters

handle

[IN] Handle.

cbOutput

[IN][OUT] Image data callback function.

```
void (__stdcall* cbOutput)(
  unsigned char               *pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX2   *pstFrameInfo,
  void                        *pUser
)
```

pData

[IN][OUT] Image data.

pstFrameInfo

[IN][OUT] Obtained frame information (including bar code quality information), see ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** for details.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx,

MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.

- This API supports getting the bar code quality information of ID5000-series smart code reader. While for the code readers without bar code scoring function, the obtained information of bar code quality is 0.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.4.6 MV_CODEREADER_RegisterTriggerCallBack

Register the callback function for trigger information, which is provided by the device firmware. You can call this API after the device is connected. This API is not supported by virtual code readers.

API Definition

```
int MV_CODEREADER_RegisterTriggerCallBack(
  void        *handle,
  void        (__stdcall* cbTrigger)(MV_CODEREADER_TRIGGER_INFO_DATA* pstTriggerInfo, void* pUser),
  void        *pUser
);
```

Parameters

handle

[IN] Device handle.

cbTrigger

[IN][OUT] Pointer to the trigger callback function.

```
void (__stdcall* cbTrigger)(
  MV_CODEREADER_TRIGGER_INFO_DATA    *pstTriggerInfo,
  void                               *pUser
)
```

pstTriggerInfo

[IN][OUT] Trigger information. See ***MV_CODEREADER_TRIGGER_INFO_DATA*** for details.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.4.7 MV_CODEREADER_MSC_RegisterImageCallBack

Register the callback function for image data of a specified channel. You can get the image information including the bar code quality information through this API. This API is supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_MSC_RegisterImageCallBack(
  void            *handle,
  unsigned int      nChannelID,
  void            (__stdcall* cbOutput)(unsigned char * pData, MV_CODEREADER_IMAGE_OUT_INFO_EX2*
pstFrameInfo, void* pUser),
  void            *pUser
);
```

Parameters

handle

[IN] Handle.

nChannelID

[IN] Channel ID. For single-channel firmwares, the channel ID is 0; for multi-channel firmwares, the channel ID starts from 0, and the number of channels depends on the number of sensors. For example, if the number of sensors is 3, the channel ID range is: [0,1,2].

cbOutput

[IN][OUT] Image data callback function.

```
void (__stdcall* cbOutput)(
  unsigned char               *pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX2   *pstFrameInfo,
  void                        *pUser
)
```

pData

[IN][OUT] Image data.

pstFrameInfo

[IN][OUT] Obtained frame information, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** for details.

pUser

[IN][OUT] User pointer.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout, MV_CODEREADER_GetOneFrameTimeoutEx, MV_CODEREADER_GetOneFrameTimeoutEx2, MV_CODEREADER_RegisterImageCallBack, MV_CODEREADER_RegisterImageCallBackEx, MV_CODEREADER_RegisterImageCallBackEx2, MV_CODEREADER_MSC_GetOneFrameTimeout, and MV_CODEREADER_MSC_RegisterImageCallBack.
- This API supports getting image information of different channels for IDX-series smart code reader.
- This API supports getting image information of singe-channel firmwares and the input parameter **nChannelID** should be set to 0.
- This API supports getting the bar code quality information of ID5000-series smart code reader. While for the code readers without bar code scoring function, the information of bar code quality is 0.
- To stop or cancel the callback of a specified channel, you can just set the parameter **cbOutput** to NULL.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.5 Parameters Settings APIs

This section introduces APIs of getting and setting the code reader parameters.

Note

Parts of APIs are supported by the virtual code reader. For the parameters configuration of the virtual code reader, when calling an API succeeded, a success will always be returned, but only the configuration of those supported parameters will take effect.

3.5.1 MV_CODEREADER_GetIntValue

Get the property value of integer type.

API Definition

```
int MV_CODEREADER_GetIntValue(
  void                    *handle,
  const char              *strKey,
  MV_CODEREADER_INTVALUE_EX  *pIntValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pIntValue

[IN][OUT] Device parameter list, which contains the maximum values, minimum values, and current values of the parameters. See ***MV_CODEREADER_INTVALUE_EX*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.2 MV_CODEREADER_SetIntValue

Set the property value of integer type.

API Definition

```
int MV_CODEREADER_SetIntValue(
  void                    *handle,
  const char              *strKey,
  int64_t                 nValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

nValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.3 MV_CODEREADER_GetEnumValue

Get the property value of enumerated type.

API Definition

```
int MV_CODEREADER_GetEnumValue(
  void                  *handle,
  const char              *strKey,
  MV_CODEREADER_ENUMVALUE    *pEnumValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pEnumValue

[IN][OUT] Device parameter list, which contains the current values and the number of valid data. Refer to the structure ***MV_CODEREADER_ENUMVALUE*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.4 MV_CODEREADER_SetEnumValue

Set the property value of enumerated type.

API Definition

```
int MV_CODEREADER_SetEnumValue(
  void                  *handle,
  const char              *strKey,
  unsigned int             nValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

nValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.5 MV_CODEREADER_SetEnumValueByString

Set the property value of enumerated type by string.

API Definition

```
int MV_CODEREADER_SetEnumValueByString(
  void            *handle,
  const char      *strKey,
  const char      *sValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

sValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

This API is not supporte by Machine Vision Camera SDK (MVCameraSDK) with version V2.3.1 or below.

3.5.6 MV_CODEREADER_GetFloatValue

Get the property value of float type.

API Definition

```
int MV_CODEREADER_GetFloatValue(
  void                  *handle,
  const char              *strKey,
  MV_CODEREADER_FLOATVALUE   *pFloatValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pFloatValue

[IN][OUT] Device parameter list, which contains the maximum values, minimum values, and current values of the parameters. Refer to the structure ***MV_CODEREADER_FLOATVALUE*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Getting the Property Value of Float Type

```
int nRet = MV_CODEREADER_GetFloatValue(m_hDevHandle, strKey, &stParam);
if (MV_CODEREADER_OK != nRet)
{
    return nRet;
}
```

3.5.7 MV_CODEREADER_SetFloatValue

Set the property value of float type.

API Definition

```
int MV_CODEREADER_SetFloatValue(
  void                  *handle,
  const char              *strKey,
```

```
  float                 fValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

fValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.8 MV_CODEREADER_GetStringValue

Get the property value of string type.

API Definition

```
int MV_CODEREADER_GetStringValue(
  void                  *handle,
  const char            *strKey,
  MV_CODEREADER_STRINGVALUE  *pStringValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pStringValue

[IN][OUT] Device parameter value. Refer to the structure ***MV_CODEREADER_STRINGVALUE*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Getting the Property Value of String Type

```
int nRet = MV_CODEREADER_GetStringValue(m_hDevHandle, strKey, &stParam);
if (MV_CODEREADER_OK != nRet)
{
  return nRet;
}
```

3.5.9 MV_CODEREADER_SetStringValue

Set the property value of string type.

API Definition

```
int MV_CODEREADER_SetStringValue(
  void                *handle,
  const char          *strKey,
  const char          *sValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

sValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.10 MV_CODEREADER_GetBoolValue

Get the property value of boolean type.

API Definition

```
int MV_CODEREADER_GetBoolValue(
  void                *handle,
  const char          *strKey,
  bool                *pBoolValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pBoolValue

[IN][OUT] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.11 MV_CODEREADER_SetBoolValue

Set the property value of boolean type.

API Definition

```
int MV_CODEREADER_SetBoolValue(
  void              *handle,
  const char          *strKey,
  bool              bValue
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

bValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.12 MV_CODEREADER_SetCommandValue

Set the property value of command type.

API Definition

```
int MV_CODEREADER_SetCommandValue(
  void              *handle,
  const char        *strKey,
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Setting the Property Value of Command Type

```
nRet = MV_CODEREADER_SetCommandValue(m_handle, TRIGGER_SOFTWARE);
if (MV_CODEREADER_OK != nRet)
{
    cstrInfo.Format(_T("Set Software Once failed! err code:%#x"), nRet);
    MessageBox(cstrInfo);
    return ;
}
```

3.5.13 MV_CODEREADER_GetOptimalPacketSize

Get optimal packet size.

API Definition

```
int MV_CODEREADER_GetOptimalPacketSize(
  void        *handle
);
```

Parameters

handle

[IN] Handle.

Return Value

The optimal packet size.

Remarks

The API is supported only by GigE cameras.

3.5.14 MV_CODEREADER_ReadMemory

Read the memory.

API Definition

```
int MV_CODEREADER_ReadMemory(
  void          *handle,
  void          *pBuffer,
  int64_t        nAddress,
  int64_t        nLength
)
```

Parameters

handle

[IN] Handle.

pBuffer

[IN][OUT] Memory value.

nAddress

[IN] The memory address to be read. You can get the address information from Cameral.xml file, which will be generated automatically in the directory of current program after opening device.

nLength

[IN] The memory length to be read.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

If the value of **nLength** is 4, it indicates to read register; if the value of **nLength** is greater than 4, it indicates to read memory.

3.5.15 MV_CODEREADER_WriteMemory

Write to the memory.

API Definition

```
int MV_CODEREADER_WriteMemory(
  void          *handle,
  void          *pBuffer,
  int64_t        nAddress,
  int64_t        nLength
)
```

Parameters

handle

[IN] Handle.

pBuffer

[IN] Memory value to be written.

nAddress

[IN] The memory address to be written. You can get the address information from Cameral.xml file, which will be generated automatically in the directory of current program after opening device.

nLength

[IN] Memory length to be written.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

If the value of **nLength** is 4, it indicates to write to register; if the value of **nLength** is greater than 4, it indicates to write to memory.

3.6 Image Matting APIs

This section introduces APIs of getting and setting the image matting algorithm, getting waybill information, enabling the matting, and other image matting functions.

Note

The image matting APIs are not supported by the virtual code reader.

3.6.1 MV_CODEREADER_Algorithm_GetIntValue

Get matting algorithm parameters.

API Definition

```
int MV_CODEREADER_Algorithm_GetIntValue(
  void                *handle,
  const char* const       strParamKeyName,
  int* const              pnValue
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to ***Algorithm Parameter*** .

pnValue

[IN][OUT] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.6.2 MV_CODEREADER_Algorithm_SetIntValue

Set matting algorithm parameters.

API Definition

```
int MV_CODEREADER_Algorithm_SetIntValue(
  void              *handle,
  const char* const     strParamKeyName,
  const int             nValue
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to ***Algorithm Parameter*** .

nValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.6.3 MV_CODEREADER_GetWayBillInfo

Get the waybill information of input image.

API Definition

```
int MV_CODEREADER_GetWayBillInfo(
  void                          *handle,
  unsigned char                 *pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX     *pstFrameInfo
);
```

Parameters

handle

[IN] Handle.

pData

[IN] Pointer to original image.

pstFrameInfo

[IN][OUT] Image information, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.6.4 MV_CODEREADER_GetWayBillInfoEx

Get the waybill information of input image (the obtained image information contains bar code quality).

API Definition

```
int MV_CODEREADER_GetWayBillInfoEx(
  void                          *handle,
  unsigned char                 *pData,
  MV_CODEREADER_IMAGE_OUT_INFO_EX2    *pstFrameInfo
);
```

Parameters

handle

[IN] Handle.

pData

[IN] Pointer to original image

pstFrameInfo

[IN][OUT] Image information, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.6.5 MV_CODEREADER_SetWayBillEnable

Enable or disable the image matting.

API Definition

```
int MV_CODEREADER_SetWayBillEnable(
  void      *handle,
  bool      bWaybillEnable
);
```

Parameters

handle

[IN] Handle.

bWaybillEnable

[IN] Enable matting.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

You can get the waybill information only when the image matting function of code reader is enabled (related node: **DisplayBillInfoEnable**) and the value of parameter **bWaybillEnable** is true.

Chapter 4 Data Structure and Enumeration

4.1 Data Structure

4.1.1 MV_CODEREADER_BCR_INFO

Structure about Bar Code Information

Member	Data Type	Description
nID	unsigned int	Bar code ID
chCode	char	Characters, the maximum length is 256 bytes (value of macro definition "MV_CODEREADER_MAX_BCR_CODE_LEN"). If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nLen	unsigned int	The length of characters
nBarType	unsigned int	Bar code type
pt	***MV_CODEREADER_POINT_I***	Bar code location, the maximum length is 4 bytes.
nAngle	int	Bar code angle (×10), range: 0 to 3600 • 1D code: Clockwise 3600 degrees from positive X-axis of the image • 2D code: Counterclockwise 3600 degrees from positive X-axis of the image
nMainPackageId	unsigned int	Main package ID
nSubPackageId	unsigned int	Sub package ID
sAppearCount	unsigned short	The number of the bar code scanned times of one package
sPPM	unsigned short	PPM (10 times)
sAlgoCost	unsigned short	Algorithm time cost
sSharpness	unsigned short	Image definition (×10)

4.1.2 MV_CODEREADER_BCR_INFO_EX

Structure about Bar Code Information including Quality Information

Member	Data Type	Description
nID	unsigned int	Bar code ID
chCode	char	Characters, the maximum length is 256 bytes (value of macro definition "MV_CODEREADER_MAX_BCR_CODE_LEN"). If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nLen	unsigned int	The length of characters
nBarType	unsigned int	Bar code type
pt	***MV_CODEREADER_POINT_I***	Bar code location, the maximum length is 4 bytes.
stCodeQuality	***MV_CODEREADER_CODE_INFO***	Bar code quality scoring
nAngle	int	Bar code angle (×10), range: 0 to 3600 • 1D code: Clockwise 3600 degrees from positive X-axis of the image • 2D code: Counterclockwise 3600 degrees from positive X-axis of the image
nMainPackageId	unsigned int	Main package ID
nSubPackageId	unsigned int	Sub package ID
sAppearCount	unsigned short	The number of times that the bar code of one package is scanned
sPPM	unsigned short	PPM (10 times)
sAlgoCost	unsigned short	Algorithm time cost
sSharpness	unsigned short	Image definition
bIsGetQuality	bool	Whether the device supports 2D code scoring
nIDRScore	unsigned int	Code score
n1DIsGetQuality	unsigned int	Whether the device supports 1D code scoring

Member	Data Type	Description
nTotalProcCost	unsigned int	Time duration from trigger start time to app output time. Unit: millisecond.
nTriggerTimeTvHigh	unsigned int	High-order 32-bit of trigger start time (unit: second).
nTriggerTimeTvLow	unsigned int	Low-order 32-bit of trigger start time (unit: second).
nTriggerTimeUtvHigh	unsigned int	High-order 32-bit of trigger start time (unit: microsecond).
nTriggerTimeUtvLow	unsigned int	Low-order 32-bit of trigger start time (unit: microsecond).
nReserved[24]	unsigned int	Reserved.

4.1.3 MV_CODEREADER_BCR_INFO_EX2

Structure about Extended Bar Code Information Including Quality Information

Member	Data Type	Description
nID	unsigned int	Bar code ID
chCode	char	Characters, the maximum length is 4096 bytes (value of macro definition "MV_CODEREADER_MAX_BCR_CODE_LEN_EX").
nLen	unsigned int	Actual length of characters
nBarType	unsigned int	Bar code type
pt	***MV_CODEREADER_POINT_I***	Bar code location, the maximum length is 4 bytes.
stCodeQuality	***MV_CODEREADER_CODE_INFO***	Bar code quality scoring
nAngle	int	Bar code angle (×10), range: 0 to 3600 • 1D code: Clockwise 3600 degrees from positive X-axis of the image • 2D code: Counterclockwise 3600 degrees from positive X-axis of the image

Member	Data Type	Description
nMainPackageId	unsigned int	Main package ID
nSubPackageId	unsigned int	Sub package ID
sAppearCount	unsigned short	Number of times that the bar code of one package is scanned
sPPM	unsigned short	PPM (10 times)
sAlgoCost	unsigned short	Algorithm time cost
sSharpness	unsigned short	Image definition (×10)
bIsGetQuality	bool	Whether the device supports 2D code scoring
nIDRScore	unsigned int	Code score
n1DIsGetQuality	unsigned int	Whether the device supports 1D code scoring
nTotalProcCost	unsigned int	Time duration from trigger start time to app output time. Unit: millisecond.
nTriggerTimeTvHigh	unsigned int	High-order 32-bit of trigger start time (unit: second).
nTriggerTimeTvLow	unsigned int	Low-order 32-bit of trigger start time (unit: second).
nTriggerTimeUtvHigh	unsigned int	High-order 32-bit of trigger start time (unit: microsecond).
nTriggerTimeUtvLow	unsigned int	Low-order 32-bit of trigger start time (unit: microsecond).
nReserved[59]	int	Reserved.

4.1.4 MV_CODEREADER_CHUNK_DATA_CONTENT

Structure about Chunk Data Content

Member	Data Type	Description
pChunkData	unsigned char*	Chunk content
nChunkID	unsigned int	Chunk ID

Member	Data Type	Description
nChunkLen	unsigned int	Chunk length
nReserved[8]	unsigned int	Reserved.

4.1.5 MV_CODEREADER_CODE_INFO

Structure about Bar Code Quality Information

Member	Data Type	Description
nOverQuality	int	Overall quality level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nDeCode	int	Decode quality level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nSCGrade	int	Symbol contrast quality level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nModGrade	int	Module uniformity level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nFPDGrade	int	Fixed pattern damage level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nANGrade	int	Axial uniformity level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nGNGrade	int	Grid uniformity level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nUECGrade	int	Level of unused error correction (for 2D code only)

Member	Data Type	Description
		Value range: [0,4]. The higher the value, the higher the quality.
nPGHGrade	int	Horizontal print growth level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nPGVGrade	int	Vertical print growth level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
fSCScore	float	Symbol contrast quality score (for both 1D and 2D codes) Value range: [0,1]. The higher the value, the higher the quality.
fModScore	float	Module uniformity score (for both 1D and 2D codes) Value range: [0,1]. The higher the value, the higher the quality.
fFPDScore	float	Fixed pattern damage score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fAnScore	float	Axial uniformity score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fGNScore	float	Grid uniformity score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fUECScore	float	Score of unused error correction (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fPGHScore	float	Horizontal print growth score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fPGVScore	float	Vertical print growth score (for 2D code only)

Member	Data Type	Description
		Value range: [0,1]. The higher the value, the higher the quality.
nRMGrade	int	Reflectance margin quality level (for 2D code only). Value range: [0,4]. The higher the value, the higher the quality.
fRMScore	float	Reflectance margin score (for 2D code only). Value range: [0,1]. The higher the value, the higher the quality.
n1DEdgeGrade	int	Quality level of edge determination (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DMinRGrade	int	Quality level of minimum reflectance (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DMinEGrade	int	Quality level of minimum edge contrast (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DDcdGrade	int	Quality level of decodability (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DDefGrade	int	Quality level of defects (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DQZGrade	int	Quality level of quiet zone (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
f1DEdgeScore	float	Score of edge determination (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.

Member	Data Type	Description
f1DMinRScore	float	Score of minimum reflectance (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DMinEScore	float	Score of minimum edge contrast (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DDcdScore	float	Score of decodability (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DDefScore	float	Score of defects (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DQZScore	float	Score of quiet zone (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
nReserved[18]	int	Reserved.

4.1.6 MV_CODEREADER_DEVICE_INFO

Structure about Device Information

Member	Data Type	Description
nMajorVer	unsigned short	Device major version number
nMinorVer	unsigned short	Device minor version number
nMacAddrHigh	unsigned int	High MAC address
nMacAddrLow	unsigned int	Low MAC address
nTLayerType	unsigned int	Transport layer protocol type of device
nDeviceType	unsigned int	Device type
bSelectDevice	bool	For selecting device, whether it is camera of the specified series: true-yes, false-no.

Member	Data Type	Description
nReserved[2]	unsigned int	Reserved
stGigEInfo	***MV_CODEREADER_GIGE_DEVICE_INFO***	GigE device information
stUsb3VInfo	***MV_CODEREADER_USB3_DEVICE_INFO***	U3V device information

4.1.7 MV_CODEREADER_DEVICE_INFO_LIST

Structure about Device List

Member	Data Type	Description
nDeviceNum	unsigned int	The number of online devices.
pDeviceInfo	***MV_CODEREADER_DEVICE_INFO*** *	Device information, up to 256 devices can be supported, which is defined by the macro definition "MV_CODEREADER_MAX_DEVICE_NUM".

4.1.8 MV_CODEREADER_ENUMVALUE

Structure about Enumeration Type Value

Member	Data Type	Description
nCurValue	unsigned int	Current value
nSupportedNum	unsigned int	The number of valid data.
nSupportValue	unsigned int	Supported enumeration types, the maximum length is 64 bytes (value of macro definition "MV_CODEREADER_MAX_XML_SYMBOLIC_NUM")
nReserved[4]	unsigned int	Reserved.

4.1.9 MV_CODEREADER_EVENT_OUT_INFO

Structure about Event Information

Member	Data Type	Description
EventName	char	Event name, the maximum length is 128 bytes (value of macro definition " MV_CODEREADER_MAX_EVENT_NAME_SIZE ").
nEventID	unsigned short	Event ID
nStreamChannel	unsigned short	Channel ID
nBlockIdHigh	unsigned int	Frame high bits
nBlockIdLow	unsigned int	Frame low bits
nTimestampHigh	unsigned int	Timestamp, high bits
nTimestampLow	unsigned int	Timestamp, low bits
pEventData	void*	Event data
nEventDataSize	unsigned int	Event data length
nReserved[16]	unsigned int	Reserved.

4.1.10 MV_CODEREADER_FILE_ACCESS

Structure about File Access

Member	Data Type	Description
pUserFileName	const char*	User file name
pDevFileName	const char*	Device file name
nReserved[32]	unsigned int	Reserved.

4.1.11 MV_CODEREADER_FILE_ACCESS_PROGRESS

Structure about File Access Progress

Member	Data Type	Description
nCompleted	int64_t	Completed size
nTotal	int64_t	Total size
nReserved[8]	unsigned int	Reserved.

4.1.12 MV_CODEREADER_FLOATVALUE

Structure about Float Type Value

Member	Data Type	Description
fCurValue	float	Current value
fMax	float	The maximum value
fMin	float	The minimum value
nReserved[4]	unsigned int	Reserved.

4.1.13 MV_CODEREADER_FRAME_OUT_INFO

Structure about Output Frame Information

Member	Data Type	Description
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
enPixelType	enum ***MvCodeReaderGvspPixelType***	Pixel format
nFrameNum	unsigned int	Frame No
nDevTimeStampHigh	unsigned int	Timestamp, high 32 bits
nDevTimeStampLow	unsigned int	Timestamp, low 32 bits
nReserved0	unsigned int	Reserved, 8-byte aligned.

Member	Data Type	Description
nHostTimeStamp	int64_t	Host-generated timestamp
nFrameLen	unsigned int	Frame length
nLostPacket	unsigned int	The number of packets lost in the current frame.
nReserved[2]	unsigned int	Reserved.

4.1.14 MV_CODEREADER_FRAME_OUT_INFO_EX

Structure about Output Frame Information

Member	Data Type	Description
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
enPixelType	enum ***MvCodeReaderGvspPixelType***	Pixel format
nFrameNum	unsigned int	Frame No
nDevTimeStampHigh	unsigned int	Timestamp, high 32 bits
nDevTimeStampLow	unsigned int	Timestamp, low 32 bits
nReserved0	unsigned int	Reserved, 8-byte aligned.
nHostTimeStamp	int64_t	Host-generated timestamp
nFrameLen	unsigned int	Frame length
nSecondCount	unsigned int	Seconds
nCycleCount	unsigned int	Cycle count
nCycleOffset	unsigned int	Cycle count offset
fGain	float	Gain
fExposureTime	float	Exposure time
nAverageBrightness	unsigned int	Average brightness
nRed	unsigned int	Red data

Member	Data Type	Description
nGreen	unsigned int	Green data
nBlue	unsigned int	Blue data
nFrameCounter	unsigned int	Image counting
nTriggerIndex	unsigned int	Trigger counting
nInput	unsigned int	Input
nOutput	unsigned int	Output
nOffsetX	unsigned short	ROI x-offset
nOffsetY	unsigned short	ROI y-offset
nChunkWidth	unsigned short	Chunk width
nChunkHeight	unsigned short	Chunk height
nLostPacket	unsigned int	The number of packets lost in the current frame.
nUnparsedChunkNum	unsigned int	Number of unparsed chunk data items.
UnparsedChunkList	union { ***MV_CODEREADER_CHUNK_DATA_CONTENT*** * pUnparsedChunkContent; int64_t nAligning; }	Unparsed chunk data list. **pUnparsedChunkContent**: structure about chunk data content
nReserved[39]	unsigned int	Reserved.

4.1.15 MV_CODEREADER_GIGE_DEVICE_INFO

Structure about GigE Device Information

Member	Data Type	Description
nIpCfgOption	unsigned int	IP type supported by device.
nIpCfgCurrent	unsigned int	Current IP type
nCurrentIp	unsigned int	Current IP address
nCurrentSubNetMask	unsigned int	Current subnet mask

Member	Data Type	Description
nDefultGateWay	unsigned int	Default gateway
chManufacturerName[32]	unsigned char	Device manufacturer
chModelName[32]	unsigned char	Device model
chDeviceVersion[32]	unsigned char	Device version
chManufacturerSpecificInfo[48]	unsigned char	Special information of manufacturer
chSerialNumber[16]	unsigned char	Device serial number
chUserDefinedName[16]	unsigned char	Custom name
nNetExport	unsigned int	Host port IP address
nCurUserIP	unsigned int	The IP of the user occupying the current device. Not supported by all device models. **Note** This member is valid for the RunTime package with version 3.5.1 and above.
nReserved[3]	unsigned int	Reserved.

See Also

MV_CODEREADER_DEVICE_INFO

4.1.16 MV_CODEREADER_IMAGE_OUT_INFO

Structure about Output Frame Information Which Contains Bar Code Information

Member	Data Type	Description
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
enPixelType	***MvCodeReaderGvspPixelType***	Pixel or image format
nTriggerIndex	unsigned int	Trigger index. Only valid when level is triggered.
nFrameNum	unsigned int	Frame No.

Member	Data Type	Description
nFrameLen	unsigned int	Current frame size
nTimeStampHigh	unsigned int	High 32-bit timestamp
nTimeStampLow	unsigned int	Low 32-bit timestamp
nResultType	unsigned int	Output message type
chResult	unsigned char	Convert to different structures depending on the message type. The maximum size is 1024×64 (defined by macro definition "MV_CODEREADER_MAX_RESULT_SIZE").
bIsGetCode	bool	Whether the bar code is read.
pImageWaybill	unsigned char*	Waybill image
nImageWaybillLen	unsigned int	Waybill data size
bFlaseTrigger	unsigned int	Whether triggered by mistake.
nFocusScore	unsigned int	Focus score
nChannelID	unsigned int	Channel ID corresponding to the stream
nImageCost	unsigned int	The algorithm processing time cost for recognizing the information contained in the image (unit: millisecond)
nWholeFlag	unsigned short	Marker of complete image: 1
nRes	unsigned short	Reserved.
nReserved[5]	unsigned int	Reserved.

4.1.17 MV_CODEREADER_IMAGE_OUT_INFO_EX

Structure about Output Frame Information Which Contains Bar Code and Waybill Information

Member	Data Type	Description
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
enPixelType	***MvCodeReaderGvspPixelType***	Pixel or image format

Member	Data Type	Description
nTriggerIndex	unsigned int	Trigger index. Only valid when level is triggered.
nFrameNum	unsigned int	Frame No.
nFrameLen	unsigned int	Current frame size
nTimeStampHigh	unsigned int	High 32-bit timestamp
nTimeStampLow	unsigned int	Low 32-bit timestamp
bFlaseTrigger	unsigned int	Whether triggered by mistake.
nFocusScore	unsigned int	Focus score
bIsGetCode	bool	Whether the bar code is read.
pstCodeList	***MV_CODEREADER_RESULT_BCR*** *	Bar code list
pstWaybillList	***MV_CODEREADER_WAYBILL_LIST*** *	Waybill information
nEventID	unsigned int	Event ID
nChannelID	unsigned int	Channel ID corresponding to the stream
nImageCost	unsigned int	The algorithm processing time cost for recognizing the information contained in the image (unit: millisecond)
UnparsedOcrList	union { ***MV_CODEREADER_OCR_INFO_LIST*** * pstOcrList; int64_t nAligning;}	Unparsed OCR information list **pstOcrList**: OCR information
nWholeFlag	unsigned short	Marker of complete image: 1
nRes	unsigned short	Reserved.
nReserved[3]	unsigned int	Reserved.

4.1.18 MV_CODEREADER_IMAGE_OUT_INFO_EX2

Structure about Output Frame Information Which Contains Bar Code (with QR Code) and Waybill Information

Member	Data Type	Description
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
enPixelType	***MvCodeReaderGvspPixelType***	Pixel or image format
nTriggerIndex	unsigned int	Trigger index. Valid only when level is triggered.
nFrameNum	unsigned int	Frame No.
nFrameLen	unsigned int	Current frame size
nTimeStampHigh	unsigned int	High 32-bit timestamp
nTimeStampLow	unsigned int	Low 32-bit timestamp
bFlaseTrigger	unsigned int	Whether triggered by mistake.
nFocusScore	unsigned int	Focus score
bIsGetCode	bool	Whether the bar code is read.
pstCodeListEx	***MV_CODEREADER_RESULT_BCR_EX*** *	Bar code list
pstWaybillList	***MV_CODEREADER_WAYBILL_LIST*** *	Waybill information
nEventID	unsigned int	Event ID
nChannelID	unsigned int	Channel ID corresponding to the stream
nImageCost	unsigned int	The algorithm processing time cost for recognizing the information contained in the image (unit: millisecond)
UnparsedBcrList	union { ***MV_CODEREADER_RESULT_BCR_EX2*** * pstCodeListEx2; int64_t nAligning; }	Unparsed bar code list. **pstCodeListEx2**: structure about extended bar code information list. It is recommended to use this structure to parse bar code information.
UnparsedOcrList	union { ***MV_CODEREADER_OCR_INFO_LIST*** * pstOcrList;	Unparsed OCR information list **pstOcrList**: OCR information

Member	Data Type	Description
	int64_t nAligning;}	
nWholeFlag	unsigned short	Marker of complete image: 1
nRes	unsigned short	Reserved.
nReserved[25]	unsigned int	Reserved.

Remarks

Compared with structure ***MV_CODEREADER_IMAGE_OUT_INFO_EX*** , this structure contains the bar code quality information.

4.1.19 MV_CODEREADER_INTVALUE_EX

Structure about Integer Type Value

Member	Data Type	Description
nCurValue	int64_t	Current value
nMax	int64_t	The maximum value
nMin	int64_t	The minimum value
nInc	int64_t	Increment
nReserved[16]	unsigned int	Reserved.

4.1.20 MV_CODEREADER_OCR_INFO_LIST

Structure about OCR Information List

Member	Data Type	Description
nOCRAllNum	unsigned int	The total number of OCR rows of all waybills
stOcrRowInfo	***MV_CODEREADER_OCR_ROW_INFO***	OCR row basic information, the maximum length is 100 bytes (value of macro definition "MAX_ CODEREADER_OCR_COUNT")
nReserved[8]	int	Reserved.

4.1.21 MV_CODEREADER_OCR_ROW_INFO

Structure about OCR Basic Information

Member	Data Type	Description
nID	unsigned int	OCR ID
nOcrLen	unsigned int	OCR characters actual length.
chOcr	char	Recognized OCR characters. The maximum length is 128 bytes (value of macro definition "MV_CODEREADER_MAX_OCR_LEN")
fCharConfidence	float	The characters confidence
nOcrRowCenterX	unsigned int	The center point X-coordinate of one row OCR
nOcrRowCenterY	unsigned int	The center point Y-coordinate of one row OCR
nOcrRowWidth	unsigned int	The width of one row OCR rectangle
nOcrRowHeight	unsigned int	The height of one row OCR rectangle
fOcrRowAngle	float	The angle of one OCR row rectangle
fDeteConfidence	float	The confidence of one row OCR location
sOcrAlgoCost	unsigned short	The OCR algorithm time cost, unit: millisecond
sReserved	unsigned short	Reserved.
nReserved[31]	int	Reserved.

4.1.22 MV_CODEREADER_POINT_F

Structure about Float Coordinates

Member	Data Type	Description
x	float	X-coordinate
y	float	Y-coordinate

4.1.23 MV_CODEREADER_POINT_I

Structure about Integer Coordinates

Member	Data Type	Description
x	int	X-coordinate
y	int	Y-coordinate

4.1.24 MV_CODEREADER_RESULT_BCR

Structure about Bar Code List Information

Member	Data Type	Description
nCodeNum	unsigned int	The number of bar codes
stBcrInfo	***MV_CODEREADER_BCR_INFO***	Bar code information, the maximum number of bar codes is 200 (value of macro definition "MAX_CODEREADER_BCR_COUNT") If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nReserved[4]	unsigned int	Reserved.

4.1.25 MV_CODEREADER_RESULT_BCR_EX

Structure about Bar Code List Information including Bar Code Quality

Member	Data Type	Description
nCodeNum	unsigned int	The number of bar codes
stBcrInfoEx	***MV_CODEREADER_BCR_INFO_EX***	Bar code information, the maximum number of bar codes is 200 (value of macro definition "MAX_CODEREADER_BCR_COUNT") If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nReserved[4]	unsigned int	Reserved.

4.1.26 MV_CODEREADER_RESULT_BCR_EX2

Structure about Extended Bar Code Information List

Member	Data Type	Description
nCodeNum	unsigned int	Number of bar codes
stBcrInfoEx2	***MV_CODEREADER_BCR_INFO_EX2***	Bar code information, the maximum number of bar codes is 300 (value of macro definition "MAX_CODEREADER_BCR_COUNT_EX")
nReserved[8]	unsigned int	Reserved.

4.1.27 MV_CODEREADER_SAVE_IMAGE_PARAM_EX

Structure about Image Saving Parameters

Member	Data Type	Description
pData	unsigned char*	Input data buffer
nDataLen	unsigned int	Input data size
enPixelType	enum ***MvCodeReaderGvspPixelType***	Pixel format of input data

Member	Data Type	Description
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
pImageBuffer	unsigned char*	Output image buffer
nImageLen	unsigned int	Output image size
nBufferSize	unsigned int	Size of provided output buffer area.
enImageType	enum ***MV_CODEREADER_IAMGE_TYPE***	Output image format
nJpgQuality	unsigned int	Encoding quality, range: (50,99]
iMethodValue	unsigned int	Interpolation method to convert Bayer to RGB24: 0-nearest neighbor, 1-bilinearity, 2-Hamilton, others-nearest neighbor.
nReserved[3]	unsigned int	Reserved.

4.1.28 MV_CODEREADER_STRINGVALUE

Structure about String Type Value

Member	Data Type	Description
chCurValue	char	Current value, the maximum length is 256 bytes.
nMaxLength	int64_t	The maximum length
nReserved[2]	unsigned int	Reserved.

4.1.29 MV_CODEREADER_TRIGGER_INFO_DATA

Structure about Trigger Information

Member	Data Type	Description
nTriggerIndex	unsigned int	Trigger No., i.e., synchronous trigger No.
nTriggerFlag	unsigned int	Trigger status: 1 (start), 0 (end).

Member	Data Type	Description
nTriggerTimeHigh	unsigned int	High-order 4-bit of trigger time.
nTriggerTimeLow	unsigned int	Low-order 4-bit of trigger time.
nOriginalTrigger	unsigned int	Original trigger No. (code reader's trigger No.)
nIsForceOver	unsigned short	Whether it is forced to end: 0 (no, it ends normally), 1 (yes, it is forced to end). This parameter is the code reader internal mechanism and cannot be configured.
nIsMainCam	unsigned short	Whether it is main code reader of sub code reader: 1 (main code reader), 0 (sub code reader).
nHostTimeStamp	int64_t	Timestamp of main code reader.
reserved[30]	unsigned int	Reserved.

4.1.30 MV_CODEREADER_USB3_DEVICE_INFO

U3V device information structure.

Structure about U3V Device Information

Member	Data Type	Description
CrtlInEndPoint	unsigned char	Control input endpoint
CrtlOutEndPoint	unsigned char	Control output endpoint
StreamEndPoint	unsigned char	Stream endpoint
EventEndPoint	unsigned char	Event endpoint
idVendor	unsigned short	Supplier ID
idProduct	unsigned short	Device ID
nDeviceNumber	unsigned int	Device No.
chDeviceGUID	unsigned char	Device GUID number, the length is defined by the macro definition "INFO_MAX_BUFFER_SIZE" (the value is 64).

Member	Data Type	Description
chVendorName	unsigned char	Supplier name, the length is defined by the macro definition "INFO_MAX_BUFFER_SIZE" (the value is 64).
chModelName	unsigned char	Device model, the length is defined by the macro definition "INFO_MAX_BUFFER_SIZE" (the value is 64).
chFamilyName	unsigned char	Device family name, the length is defined by the macro definition "INFO_MAX_BUFFER_SIZE" (the value is 64).
chDeviceVersion	unsigned char	Device version, the length is defined by the macro definition "INFO_MAX_BUFFER_SIZE" (the value is 64).
chManufacturerName	unsigned char	Device manufacturer, the length is defined by the macro definition "INFO_MAX_BUFFER_ SIZE" (the value is 64).
chSerialNumber	unsigned char	Device serial number, the length is defined by the macro definition "INFO_MAX_BUFFER_ SIZE" (the value is 64).
chUserDefinedName	unsigned char	User-defined name, the length is defined by the macro definition "INFO_MAX_BUFFER_SIZE" (the value is 64).
nbcdUSB	unsigned int	Supported USB protocols
nReserved[3]	unsigned int	Reserved.

See Also

MV_CODEREADER_DEVICE_INFO

4.1.31 MV_CODEREADER_WAYBILL_INFO

Structure about Waybill Information

Member	Data Type	Description
fCenterX	float	Center point x-coordinate
fCenterY	float	Center point y-coordinate

Member	Data Type	Description
fWidth	float	Rectangle width, which is major semi-axis
fHeight	float	Rectangle height, which is minor semi-axis
fAngle	float	Rectangle angle
fConfidence	float	Confidence
pImageWaybill	unsigned char*	Waybill image
nImageLen	unsigned int	Image length
nOcrRowNum	unsigned int	The number of OCR rows of current waybill
nReserved[11]	unsigned int	Reserved.

4.1.32 MV_CODEREADER_WAYBILL_LIST

Structure about Waybill Information List

Member	Data Type	Description
nWaybillNum	unsigned int	The number of waybills
enWaybillImageType	***MV_CODEREADER_IAMGE_TYPE***	Waybill image type
stWaybillInfo	***MV_CODEREADER_WAYBILL_INFO***	Waybill information, the maximum length is 50 bytes (value of macro definition "MAX_CODEREADER_WAYBILL_COUNT")
nOcrAllNum	unsigned int	The total number of OCR rows of all waybills
nReserved[3]	unsigned int	Reserved.

4.2 Enumeration

4.2.1 MV_CODEREADER_IAMGE_TYPE

Enumeration about Image Format

Enumeration Type	Value	Description
MV_CODEREADER_Image_ Undefined	0	Undefined format
MV_CODEREADER_Image_Mono8	1	MONO8
MV_CODEREADER_Image_Jpeg	2	JPEG
MV_CODEREADER_Image_Bmp	3	BMP
MV_CODEREADER_Image_RGB24	4	RGB24
MV_CODEREADER_Image_Png	5	PNG image, which is not supported now.
MV_CODEREADER_Image_Tif	6	TIF image, which is not supported now.

4.2.2 MvCodeReaderGvspPixelType

Pixel format enumeration.

Enumeration Definition

```
typedef MvCodeReaderGvspPixelType{
  //Undefined pixel format
  PixelType_CodeReader_Gvsp_Undefined             =  0xFFFFFFFF,
  //Mono buffer format
  PixelType_CodeReader_Gvsp_Mono1p                =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(1) | 0x0037),
  PixelType_CodeReader_Gvsp_Mono2p                =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(2) | 0x0038),
  PixelType_CodeReader_Gvsp_Mono4p                =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(4) | 0x0039),
  PixelType_CodeReader_Gvsp_Mono8                 =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(8) | 0x0001),
  PixelType_CodeReader_Gvsp_Mono8_Signed          =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(8)  | 0x0002),
  PixelType_CodeReader_Gvsp_Mono10                =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0003),
  PixelType_CodeReader_Gvsp_Mono10_Packed         =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0004),
  PixelType_CodeReader_Gvsp_Mono12                =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0005),
  PixelType_CodeReader_Gvsp_Mono12_Packed         =  (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0006),
```

```
  PixelType_CodeReader_Gvsp_Mono14              = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0025),
  PixelType_CodeReader_Gvsp_Mono16              = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0007),
  //Bayer buffer format
  PixelType_CodeReader_Gvsp_BayerGR8            = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(8) | 0x0008),
  PixelType_CodeReader_Gvsp_BayerRG8            = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(8) | 0x0009),
  PixelType_CodeReader_Gvsp_BayerGB8            = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(8) | 0x000A),
  PixelType_CodeReader_Gvsp_BayerBG8            = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(8) | 0x000B),
  PixelType_CodeReader_Gvsp_BayerGR10           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x000C),
  PixelType_CodeReader_Gvsp_BayerRG10           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x000D),
  PixelType_CodeReader_Gvsp_BayerGB10           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x000E),
  PixelType_CodeReader_Gvsp_BayerBG10           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x000F),
  PixelType_CodeReader_Gvsp_BayerGR12           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0010),
  PixelType_CodeReader_Gvsp_BayerRG12           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0011),
  PixelType_CodeReader_Gvsp_BayerGB12           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0012),
  PixelType_CodeReader_Gvsp_BayerBG12           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0013),
  PixelType_CodeReader_Gvsp_BayerGR10_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0026),
  PixelType_CodeReader_Gvsp_BayerRG10_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0027),
  PixelType_CodeReader_Gvsp_BayerGB10_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0028),
  PixelType_CodeReader_Gvsp_BayerBG10_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0029),
  PixelType_CodeReader_Gvsp_BayerGR12_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x002A),
  PixelType_CodeReader_Gvsp_BayerRG12_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x002B),
  PixelType_CodeReader_Gvsp_BayerGB12_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x002C),
  PixelType_CodeReader_Gvsp_BayerBG12_Packed      = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x002D),
  PixelType_CodeReader_Gvsp_BayerGR16           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x002E),
  PixelType_CodeReader_Gvsp_BayerRG16           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x002F),
  PixelType_CodeReader_Gvsp_BayerGB16           = (MV_CODEREADER_GVSP_PIX_MONO |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0030),
  PixelType_CodeReader_Gvsp_BayerBG16           = (MV_CODEREADER_GVSP_PIX_MONO |
```

```
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0031),
  //RGB Packed buffer format defines
  PixelType_CodeReader_Gvsp_RGB8_Packed           = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x0014),
  PixelType_CodeReader_Gvsp_BGR8_Packed           = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x0015),
  PixelType_CodeReader_Gvsp_RGBA8_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(32) | 0x0016),
  PixelType_CodeReader_Gvsp_BGRA8_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(32) | 0x0017),
  PixelType_CodeReader_Gvsp_RGB10_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x0018),
  PixelType_CodeReader_Gvsp_BGR10_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x0019),
  PixelType_CodeReader_Gvsp_RGB12_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x001A),
  PixelType_CodeReader_Gvsp_BGR12_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x001B),
  PixelType_CodeReader_Gvsp_RGB16_Packed          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x0033),
  PixelType_CodeReader_Gvsp_RGB10V1_Packed        = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(32) | 0x001C),
  PixelType_CodeReader_Gvsp_RGB10V2_Packed        = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(32) | 0x001D),
  PixelType_CodeReader_Gvsp_RGB12V1_Packed        = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(36) | 0X0034),
  PixelType_CodeReader_Gvsp_RGB565_Packed         = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0035),
  PixelType_CodeReader_Gvsp_BGR565_Packed         = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0X0036),
  //YUV Packed buffer format defines
  PixelType_CodeReader_Gvsp_YUV411_Packed         = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x001E),
  PixelType_CodeReader_Gvsp_YUV422_Packed         = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x001F),
  PixelType_CodeReader_Gvsp_YUV422_YUYV_Packed    = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0032),
  PixelType_CodeReader_Gvsp_YUV444_Packed         = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x0020),
  PixelType_CodeReader_Gvsp_YCBCR8_CBYCR          = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x003A),
  PixelType_CodeReader_Gvsp_YCBCR422_8            = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x003B),
  PixelType_CodeReader_Gvsp_YCBCR422_8_CBYCRY     = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0043),
  PixelType_CodeReader_Gvsp_YCBCR411_8_CBYYCRYY   = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x003C),
  PixelType_CodeReader_Gvsp_YCBCR601_8_CBYCR      = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x003D),
  PixelType_CodeReader_Gvsp_YCBCR601_422_8        = (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x003E),
  PixelType_CodeReader_Gvsp_YCBCR601_422_8_CBYCRY = (MV_CODEREADER_GVSP_PIX_COLOR |
```

```
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0044),
 PixelType_CodeReader_Gvsp_YCBCR601_411_8_CBYYCRYY   =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x003F),
 PixelType_CodeReader_Gvsp_YCBCR709_8_CBYCR        =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x0040),
 PixelType_CodeReader_Gvsp_YCBCR709_422_8          =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0041),
 PixelType_CodeReader_Gvsp_YCBCR709_422_8_CBYCRY     =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(16) | 0x0045),
 PixelType_CodeReader_Gvsp_YCBCR709_411_8_CBYYCRYY   =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(12) | 0x0042),
 //RGB Planar buffer format
 PixelType_CodeReader_Gvsp_RGB8_Planar            =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x0021),
 PixelType_CodeReader_Gvsp_RGB10_Planar           =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x0022),
 PixelType_CodeReader_Gvsp_RGB12_Planar           =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x0023),
 PixelType_CodeReader_Gvsp_RGB16_Planar           =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(48) | 0x0024),
 //Custom format
 PixelType_CodeReader_Gvsp_Jpeg                =  (MV_CODEREADER_GVSP_PIX_CUSTOM |
MV_CODEREADER_PIXEL_BIT_COUNT(24) | 0x0001),
 PixelType_CodeReader_Gvsp_Coord3D_ABC32f          =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(96) | 0x00C0),
 PixelType_CodeReader_Gvsp_Coord3D_ABC32f_Planar     =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(96) | 0x00C1),
 PixelType_CodeReader_Gvsp_Coord3D_AC32f          =  (MV_CODEREADER_GVSP_PIX_COLOR |
MV_CODEREADER_PIXEL_BIT_COUNT(40) | 0x00C2),
 PixelType_CodeReader_Gvsp_COORD3D_DEPTH_PLUS_MASK   =  (0x82000000 |
MV_CODEREADER_PIXEL_BIT_COUNT(28) | 0x0001),
};
```

4.2.3 MV_CODEREADER_CODE_TYPE

Enumeration about Bar Code Type

Enumeration Type	Value	Description
MV_CODEREADER_CODE_NONE	0	No recognizable bar code
2D Code		
MV_CODEREADER_TDCR_DM	1	DM code
MV_CODEREADER_TDCR_QR	2	QR code
MV_CODEREADER_TDCR_MICROQR	140	MicroQR code
1D Code		

Enumeration Type	Value	Description
MV_CODEREADER_BCR_EAN8	8	EAN8 code
MV_CODEREADER_BCR_UPCE	9	UPCE code
MV_CODEREADER_BCR_UPCA	12	UPCA code
MV_CODEREADER_BCR_EAN13	13	EAN13 code
MV_CODEREADER_BCR_ISBN13	14	ISBN13 code
MV_CODEREADER_BCR_CODABAR	20	Codebar
MV_CODEREADER_BCR_ITF25	25	Interleaved 2 of 5 code
MV_CODEREADER_BCR_MATRIX25	26	MATRIX25 code
MV_CODEREADER_BCR_ITF14	27	ITF-14 code
MV_CODEREADER_BCR_MSI	30	MSI code
MV_CODEREADER_BCR_CODE11	31	Code 11
MV_CODEREADER_BCR_ INDUSTRIAL25	32	INDUSTRIAL25 code
MV_CODEREADER_BCR_CHINAPOST	33	CHINAPOST code
MV_CODEREADER_BCR_ PHARMACODE	36	Pharmacode code
MV_CODEREADER_BCR_ PHARMACODE2D	37	Pharmacode two-track code
MV_CODEREADER_BCR_CODE39	39	Code 39
MV_CODEREADER_BCR_CODE93	93	Code 93
MV_CODEREADER_BCR_CODE128	128	Code 128
MV_CODEREADER_TDCR_PDF417	131	PDF417 code
MV_CODEREADER_TDCR_AZTEC	132	AZTEC code
MV_CODEREADER_TDCR_ECC140	133	ECC140 code
MV_CODEREADER_TDCR_HANXIN	145	Hanxin code

4.2.4 MV_CODEREADER_TRIGGER_SOURCE

Enumeration about Trigger Source

Enumeration Type	Value	Description
MV_CODEREADER_TRIGGER_SOURCE_LINE0	0	Line0
MV_CODEREADER_TRIGGER_SOURCE_LINE1	1	Line1
MV_CODEREADER_TRIGGER_SOURCE_LINE2	2	Line2
MV_CODEREADER_TRIGGER_SOURCE_LINE3	3	Line3
MV_CODEREADER_TRIGGER_SOURCE_ COUNTER0	4	Counter zero trigger
MV_CODEREADER_TRIGGER_SOURCE_ TCPSERVERSTART	5	TCP server trigger
MV_CODEREADER_TRIGGER_SOURCE_ UDPSTART	6	UDP trigger
MV_CODEREADER_TRIGGER_SOURCE_ SOFTWARE	7	Soft trigger
MV_CODEREADER_TRIGGER_SOURCE_ SERIALSTART	8	Serial port trigger
MV_CODEREADER_TRIGGER_SOURCE_ SELFTRIGGER	9	Self trigger
MV_CODEREADER_TRIGGER_SOURCE_ MAINSUB	10	Main-sub trigger
MV_CODEREADER_TRIGGER_SOURCE_ TCPCLIENTSTART	12	TCP client trigger

Appendix A. Macro Definition

Device Type

Macro Definition	Value	Description
MV_CODEREADER_UNKNOW_DEVICE	0x00000000	Unknown device type
MV_CODEREADER_GIGE_DEVICE	0x00000001	GigE device
MV_CODEREADER_1394_DEVICE	0x00000002	1394-a/b device
MV_CODEREADER_USB_DEVICE	0x00000004	USB3.0 device
MV_CODEREADER_CAMERALINK_DEVICE	0x00000008	CameraLink device

IP Configuration Mode

Macro Definition	Value	Description
MV_CODEREADER_IP_CFG_STATIC	0x05000000	Static mode
MV_CODEREADER_IP_CFG_DHCP	0x06000000	DHCP mode
MV_CODEREADER_IP_CFG_LLA	0x04000000	LLA (Link-local address)

Device Access Mode

Macro Definition	Value	Description
MV_CODEREADER_ACCESS_Exclusive	1	Exclusive access mode, other apps are allowed to read CCP register only.
MV_CODEREADER_ACCESS_ExclusiveWithSwitch	2	Preempt the privilege of mode 5 and obtain the exclusive access to the device.
MV_CODEREADER_ACCESS_Control	3	Control access mode, other apps are allowed to read all registers.

Macro Definition	Value	Description
MV_CODEREADER_ACCESS_ ControlWithSwitch	4	Preempt the privilege of mode 5 and obtain the control access to the device.
MV_CODEREADER_ACCESS_ ControlSwitchEnable	5	Preemptible control access mode
MV_CODEREADER_ACCESS_ ControlSwitchEnableWithKey	6	Preempt the privilege of mode 5 and obtain the preemptible control access mode.
MV_CODEREADER_ACCESS_ Monitor	7	Read mode, which is applicable to the control access mode.

Output Protocol

Macro Definition	Value	Description
CommuPtlSmartSDK	1	SmartSDK protocol
CommuPtlTcpIP	2	TCP/IP protocol
CommuPtlSerial	3	Serial protocol

Exception Message Type

Macro Definition	Value	Description
MV_CODEREADER_EXCEPTION_ DEV_DISCONNECT	0x00008001	Device is disconnected.
MV_CODEREADER_EXCEPTION_ VERSION_CHECK	0x00008002	SDK version and driver version mismatched.

Other

Macro Definition	Description
MV_CODEREADER_MAX_ DEVICE_NUM	The maximum number of devices, the default value is 256.
MV_CODEREADER_MAX_XML_ SYMBOLIC_NUM	The maximum number of types, the default value is 64.
MV_CODEREADER_MAX_ RESULT_SIZE	The maximum size of data cache, the default value is 1024×64.

Macro Definition	Description
INFO_MAX_BUFFER_SIZE	The maximum length of information, the default value is 64.
MV_CODEREADER_MAX_EVENT_NAME_SIZE	The maximum length of event name, the default value is 128
MV_CODEREADER_MAX_BCR_CODE_LEN	The maximum length of bar code, the default value is 256.
MAX_CODEREADER_BCR_COUNT	The maximum number of bar codes for each output, the default value is 200.
MV_CODEREADER_MAX_BCR_CODE_LEN_EX	The maximum length of bar code (extended). The value is 4096.
MAX_CODEREADER_BCR_COUNT_EX	The maximum number of bar codes for each output (extended). The value is 300.
MAX_CODEREADER_WAYBILL_COUNT	The maximum number of waybills, the default value is 50.
MV_CODEREADER_MAX_OCR_LEN	The maximum length of OCR. The value is 128.
MAX_CODEREADER_OCR_COUNT	The maximum number of OCR per output. The value is 100.

Appendix B. Algorithm Parameter

This page provides the description and value setting suggestions of algorithm parameters that obtained from Machine Vision Software according to camera type.

Image Matting Parameter

Parameter Name	Description	Value Range
KEY_WAYBILL_ABILITY	Algorithm capability set	Waybill extract: 0x1, image enhancement: 0x2, code extract: 0x4, Box copy module: 0x8, waybill extracting module: 0x10, maximum module index: 0x3F
KEY_WAYBILL_MAX_WIDTH	The maximum width of algorithm	Range: [0,65535], default: 5472
KEY_WAYBILL_MAX_HEIGHT	The maximum height of algorithm	Range: [0,65535], default: 3648
KEY_WAYBILL_ OUTPUTIMAGETYPE	The output image format of waybill: 1-MONO8, 2-JPG, 3-BMP	Range: [1,3], default: 2
KEY_WAYBILL_JPGQUALITY	JPG coding quality	Range: [1,100], default: 80
KEY_WAYBILL_ ENHANCEENABLE	Whether to enable image enhancement	Range: [0,1], default: 0
KEY_WAYBILL_MINWIDTH	Minimum width of waybill	Range: [15,2592], default: 100
KEY_WAYBILL_MINHEIGHT	Minimum height of waybill	Range: [100,2048], default: 100
KEY_WAYBILL_MAXWIDTH	Maximum width of waybill	Minimum value: 15, default: 3072
KEY_WAYBILL_MAXHEIGHT	Maximum height of waybill	Minimum value: 10, default: 2048
KEY_WAYBILL_MORPHTIMES	Expansion times	Range: [0,10], default: 0
KEY_WAYBILL_GRAYLOW	Minimum gray value of the bar code and character grey on the waybill	Range: [0,255], default: 0

Parameter Name	Description	Value Range
KEY_WAYBILL_GRAYMID	Median gray value of waybill which is used to distinguish barcode from background	Range: [0,255], default: 70
KEY_WAYBILL_GRAYHIGH	Maximum gray value of waybill background	Range: [0,255], default: 130
KEY_WAYBILL_ BINARYADAPTIVE	Adaptive binarization	Range: [0,1], default: 1
KEY_WAYBILL_BOUNDARYROW	Expand the edge in row direction when matting waybill	Range: [0,2000], default: 0
KEY_WAYBILL_BOUNDARYCOL	Expand the edge in column direction when matting waybill	Range: [0,2000], default: 0
KEY_WAYBILL_ MAXBILLBARHEIGTHRATIO	Maximum height ratio of waybill to barcode	Range: [0,100], default: 20
KEY_WAYBILL_ MAXBILLBARWIDTHRATIO	Maximum width ratio of waybill to barcode	Range: [0,100], default: 5
KEY_WAYBILL_ MINBILLBARHEIGTHRATIO	Minimum height ratio of waybill to barcode	Range: [0,100], default: 5
KEY_WAYBILL_ MINBILLBARWIDTHRATIO	Minimum width ratio of waybill to barcode	Range: [0,100], default: 2
KEY_WAYBILL_ ENHANCEMETHOD	Enhancement method	Minimum value / default / no enhancement: 0x1, linear stretch: 0x2, histogram stretch: 0x3, histogram equalization: 0x4, brightness correction / maximum value: 0x5
KEY_WAYBILL_ ENHANCECLIPRATIOLOW	Enhance the low threshold ratio of stretching	Range: [0,100], default: 1
KEY_WAYBILL_ ENHANCECLIPRATIOHIGH	Enhance the high threshold ratio of stretching	Range: [0,100], default: 99
KEY_WAYBILL_ ENHANCECONTRASTFACTOR	Contrast ratio	Range: [0,10000], default: 100
KEY_WAYBILL_ ENHANCESHARPENFACTOR	Sharpness	Range: [0,10000], default: 0

Parameter Name	Description	Value Range
KEY_WAYBILL_ SHARPENKERNELSIZE	Size of sharpening filter core	Range: [3,15], default: 3
KEY_WAYBILL_ CODEBOUNDARYROW	Expand the edge in row direction when matting weight memo	Range: [0,2000], default: 0
KEY_WAYBILL_ CODEBOUNDARYCOL	Expand the edge in column direction when matting weight memo	Range: [0,2000], default: 0

Appendix C. Sample Codes

C.1 Get Data by Active Polling

The following sample codes show how to get image data by active polling.

```
#include <stdio.h>
#include <Windows.h>
#include <process.h>
#include <conio.h>
#include "MvCodeReaderParams.h"
#include "MvCodeReaderErrorDefine.h"
#include "MvCodeReaderCtrl.h"

bool g_bExit = false;

//  Wait for key press.
void WaitForKeyPress(void)
{
   while(!_kbhit())
   {
      Sleep(10);
   }
   _getch();
}

// Check the string type.
bool IsStrUTF8(const char* pBuffer, int size)
{
   if (size < 0)
   {
      return false;
   }

   bool IsUTF8 = true;
   unsigned char* start = (unsigned char*)pBuffer;
   unsigned char* end = (unsigned char*)pBuffer + size;
   if (NULL == start ||
      NULL == end)
   {
      return false;
   }
   while (start < end)
   {
      if (*start < 0x80) // ASCII character whose value is smaller than 0x80
      {
         start++;
      }
      else if (*start < (0xC0)) // Invalid UTF-8 character whose value is between 0x80 and 0xC0
```

```
    {
        IsUTF8 = false;
        break;
    }
    else if (*start < (0xE0)) // 2-byte UTF-8 character whose value is between 0xc0 and 0xE0
    {
        if (start >= end - 1)
        {
            break;
        }

        if ((start[1] & (0xC0)) != 0x80)
        {
            IsUTF8 = false;
            break;
        }

        start += 2;
    }
    else if (*start < (0xF0)) // 3-byte UTF-8 character whose value is between 0xE0 and 0xF0
    {
        if (start >= end - 2)
        {
            break;
        }

        if ((start[1] & (0xC0)) != 0x80 || (start[2] & (0xC0)) != 0x80)
        {
            IsUTF8 = false;
            break;
        }

        start += 3;
    }
    else
    {
        IsUTF8 = false;
        break;
    }
  }

  return IsUTF8;
}

// Convert char to Wchar.
bool Char2Wchar(const char *pStr, wchar_t *pOutWStr, int nOutStrSize)
{
  if (!pStr || !pOutWStr)
  {
    return false;
  }
```

```
    bool bIsUTF = IsStrUTF8(pStr, strlen(pStr));
    UINT nChgType = bIsUTF ? CP_UTF8 : CP_ACP;

    int iLen = MultiByteToWideChar(nChgType, 0, (LPCSTR)pStr, -1, NULL, 0);

    memset(pOutWStr, 0, sizeof(wchar_t) * nOutStrSize);

    if (iLen >= nOutStrSize)
    {
        iLen = nOutStrSize - 1;
    }

    MultiByteToWideChar(nChgType, 0, (LPCSTR)pStr, -1, pOutWStr, iLen);

    pOutWStr[iLen] = 0;

    return true;
}

// Convert Wchar to char.
bool Wchar2char(wchar_t *pOutWStr, char *pStr)
{
    if (!pStr || !pOutWStr)
    {
        return false;
    }

    int nLen =  WideCharToMultiByte(CP_ACP, 0, pOutWStr, wcslen(pOutWStr), NULL, 0, NULL, NULL);

    WideCharToMultiByte(CP_ACP, 0 , pOutWStr, wcslen(pOutWStr), pStr, nLen, NULL, NULL);

    pStr[nLen] = '\0';

    return true;
}

// Print the device information.
bool PrintDeviceInfo(MV_CODEREADER_DEVICE_INFO* pstMVDevInfo)
{
    if (NULL == pstMVDevInfo)
    {
        printf("The Pointer of pstMVDevInfo is NULL!\n");
        return false;
    }
    if (pstMVDevInfo->nTLayerType == MV_CODEREADER_GIGE_DEVICE)
    {
        int nIp1 = ((pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0xff000000) >> 24);
        int nIp2 = ((pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0x00ff0000) >> 16);
        int nIp3 = ((pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0x0000ff00) >> 8);
        int nIp4 = (pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0x000000ff);

        // Print current IP and the user defined name.
```

```
        printf("CurrentIp: %d.%d.%d.%d\n" , nIp1, nIp2, nIp3, nIp4);
        wchar_t strWchar[16] = {0};
        Char2Wchar((char*)pstMVDevInfo->SpecialInfo.stGigEInfo.chUserDefinedName, strWchar, 16);
        Wchar2char(strWchar, (char*)pstMVDevInfo->SpecialInfo.stGigEInfo.chUserDefinedName);
        printf("UserDefinedName: %s\n\n" , pstMVDevInfo->SpecialInfo.stGigEInfo.chUserDefinedName);
    }
    else if (pstMVDevInfo->nTLayerType == MV_CODEREADER_USB_DEVICE)
    {
        wchar_t strWchar[16] = {0};
        Char2Wchar((char*)pstMVDevInfo->SpecialInfo.stUsb3VInfo.chUserDefinedName, strWchar, 16);
        Wchar2char(strWchar, (char*)pstMVDevInfo->SpecialInfo.stUsb3VInfo.chUserDefinedName);
        printf("UserDefinedName: %s\n", pstMVDevInfo->SpecialInfo.stUsb3VInfo.chUserDefinedName);
        printf("Serial Number: %s\n", pstMVDevInfo->SpecialInfo.stUsb3VInfo.chSerialNumber);
        printf("Device Number: %d\n\n", pstMVDevInfo->SpecialInfo.stUsb3VInfo.nDeviceNumber);
    }
    else
    {
        printf("Not support.\n");
    }
    return true;
}

// Thread for getting stream.
static  unsigned int __stdcall WorkThread(void* pUser)
{
    int nRet = MV_CODEREADER_OK;

    MV_CODEREADER_IMAGE_OUT_INFO_EX2 stImageInfo = {0};
    memset(&stImageInfo, 0, sizeof(MV_CODEREADER_IMAGE_OUT_INFO_EX2));
    unsigned char * pData = NULL;

    while(1)
    {
        nRet = MV_CODEREADER_GetOneFrameTimeoutEx2(pUser, &pData, &stImageInfo, 1000);
        if (nRet == MV_CODEREADER_OK)
        {
            MV_CODEREADER_RESULT_BCR_EX2* stBcrResult =
(MV_CODEREADER_RESULT_BCR_EX2*)stImageInfo.UnparsedBcrList.pstCodeListEx2;
            MV_CODEREADER_OCR_INFO_LIST* stOcrResult =
(MV_CODEREADER_OCR_INFO_LIST*)stImageInfo.UnparsedOcrList.pstOcrList;

            printf("Get One Frame: nChannelID[%d] Width[%d], Height[%d], nFrameNum[%d], nTriggerIndex[%d]\n",
                stImageInfo.nChannelID, stImageInfo.nWidth, stImageInfo.nHeight, stImageInfo.nFrameNum,
stImageInfo.nTriggerIndex);

            printf("CodeNum[%d]\n", stBcrResult->nCodeNum);

            for (int i = 0; i < stBcrResult->nCodeNum; i++)
            {
                wchar_t strWchar[MV_CODEREADER_MAX_BCR_CODE_LEN_EX] = {0};
                Char2Wchar((char*)stBcrResult->stBcrInfoEx2[i].chCode, strWchar,
MV_CODEREADER_MAX_BCR_CODE_LEN_EX);
```

```
            Wchar2char(strWchar, (char*)stBcrResult->stBcrInfoEx2[i].chCode);
            printf("Get CodeInfo: CodeNum[%d] CodeEx[%s]\n", i, stBcrResult->stBcrInfoEx2[i].chCode);
        }

        printf("OcrAllNum[%d]\n", stOcrResult->nOCRAllNum);

        for (int i = 0; i < stOcrResult->nOCRAllNum; i ++)
        {
            printf("Get OcrInfo: OCRAllNum[%d] rowIndex[%d] chOcr[%s] ocrLen[%d]\r\n",
            stOcrResult->nOCRAllNum, i, stOcrResult->stOcrRowInfo[i].chOcr, stOcrResult->stOcrRowInfo[i].nOcrLen);
        }
    }
    else
    {
        printf("No data[0x%x]\n", nRet);
    }

    if(g_bExit)
    {
        break;
    }
  }
  return 0;
}


// Main processing function
int main()
{
  int nRet = MV_CODEREADER_OK;
  void* handle = NULL;
  unsigned int nThreadID = 0;
  void* hThreadHandle = NULL;

  do
  {
    // Enumerate devices.
    MV_CODEREADER_DEVICE_INFO_LIST stDeviceList;
    memset(&stDeviceList, 0, sizeof(MV_CODEREADER_DEVICE_INFO_LIST));
    nRet = MV_CODEREADER_EnumDevices( &stDeviceList, MV_CODEREADER_GIGE_DEVICE);
    if (MV_CODEREADER_OK != nRet)
    {
        printf("Enum Devices fail! nRet [0x%x]\n", nRet);
        break;
    }

    if (stDeviceList.nDeviceNum > 0)
    {
        for (unsigned int i = 0; i < stDeviceList.nDeviceNum; i++)
        {
            printf("[device %d]:\n", i);
            MV_CODEREADER_DEVICE_INFO* pDeviceInfo = stDeviceList.pDeviceInfo[i];
```

```
        if (NULL == pDeviceInfo)
        {
            break;
        }
        PrintDeviceInfo(pDeviceInfo);
    }
}
else
{
    printf("Find No Devices!\n");
    break;
}

printf("Please Intput camera index:");
unsigned int nIndex = 0;
scanf("%d", &nIndex);

if (nIndex >= stDeviceList.nDeviceNum)
{
    printf("Intput error!\n");
    break;
}

// Select the device and create a handle.
nRet = MV_CODEREADER_CreateHandle(&handle, stDeviceList.pDeviceInfo[nIndex]);
if (MV_CODEREADER_OK != nRet)
{
    printf("Create Handle fail! nRet [0x%x]\n", nRet);
    break;
}

// Open the device.
nRet = MV_CODEREADER_OpenDevice(handle);
if (MV_CODEREADER_OK != nRet)
{
    printf("Open Device fail! nRet [0x%x]\n", nRet);
    break;
}

// Set the trigger mode to off.
nRet = MV_CODEREADER_SetEnumValue(handle, "TriggerMode", MV_CODEREADER_TRIGGER_MODE_OFF);
if (MV_CODEREADER_OK != nRet)
{
    printf("Set Trigger Mode fail! nRet [0x%x]\n", nRet);
    break;
}

// Start acquisition (getting stream).
nRet = MV_CODEREADER_StartGrabbing(handle);
if (MV_CODEREADER_OK != nRet)
{
    printf("Start Grabbing fail! nRet [0x%x]\n", nRet);
```

```
        break;
    }

    // Start thread of getting stream.
    hThreadHandle = (void*) _beginthreadex( NULL , 0 , WorkThread , handle, 0 , &nThreadID );
    if (NULL == hThreadHandle)
    {
        printf("Start Recv Thread failed \n");
        break;
    }

    printf("Press a key to stop grabbing.\n");
    WaitForKeyPress();

    g_bExit = true;
    Sleep(100);

    // Stop acquisition.
    nRet = MV_CODEREADER_StopGrabbing(handle);
    if (MV_CODEREADER_OK != nRet)
    {
        printf("Stop Grabbing fail! nRet [0x%x]\n", nRet);
        break;
    }

    if (handle != NULL)
    {
        // Close the device.
        // Destroy the handle.
        MV_CODEREADER_CloseDevice(handle);
        MV_CODEREADER_DestroyHandle(handle);
        handle = NULL;
    }

    CloseHandle(hThreadHandle);
    hThreadHandle = NULL;

} while (0);


if (handle != NULL)
{
    MV_CODEREADER_CloseDevice(handle);
    MV_CODEREADER_DestroyHandle(handle);
    handle = NULL;
}

 CloseHandle(hThreadHandle);
 hThreadHandle = NULL;

printf("Press a key to exit.\n");
WaitForKeyPress();
```

```
    return 0;
}
```

C.2 Register Callback Function

The following sample codes show how to register the callback function for image data.

```
#include <stdio.h>
#include <Windows.h>
#include <conio.h>
#include "MvCodeReaderParams.h"
#include "MvCodeReaderErrorDefine.h"
#include "MvCodeReaderCtrl.h"
#include <iostream>
#include <process.h>
#include <list>
#include "Lock.h"

using namespace std;

typedef struct _CODEREADER_IMAGE_OUT_INFO_
{
    unsigned char* pFrameData;
    MV_CODEREADER_IMAGE_OUT_INFO_EX2 stFrameInfo;
}CODEREADER_IMAGE_OUT_INFO;

CMutex g_Lock;
list <CODEREADER_IMAGE_OUT_INFO> g_lstFrameInfoList;
bool  g_bRunState = false;
unsigned int g_nSaveNum = 0;

// Wait for key press.
void WaitForKeyPress(void)
{
    while(!_kbhit())
    {
        Sleep(10);
    }
    _getch();
}

// Check the string type.
bool IsStrUTF8(const char* pBuffer, int size)
{
    if (size < 0)
    {
        return false;
    }
```

```
bool IsUTF8 = true;
unsigned char* start = (unsigned char*)pBuffer;
unsigned char* end = (unsigned char*)pBuffer + size;
if (NULL == start ||
   NULL == end)
{
   return false;
}
while (start < end)
{
   if (*start < 0x80) // ASCII character whose value is smaller than 0x80
   {
      start++;
   }
   else if (*start < (0xC0)) // Invalid UTF-8 character whose value is between 0x80 and 0xC0
   {
      IsUTF8 = false;
      break;
   }
   else if (*start < (0xE0)) // 2-byte UTF-8 character whose value is between 0xc0 and 0xE0
   {
      if (start >= end - 1)
      {
         break;
      }

      if ((start[1] & (0xC0)) != 0x80)
      {
         IsUTF8 = false;
         break;
      }

      start += 2;
   }
   else if (*start < (0xF0)) // 3-byte UTF-8 character whose value is between 0xE0 and 0xF0
   {
      if (start >= end - 2)
      {
         break;
      }

      if ((start[1] & (0xC0)) != 0x80 || (start[2] & (0xC0)) != 0x80)
      {
         IsUTF8 = false;
         break;
      }

      start += 3;
   }
   else
   {
      IsUTF8 = false;
```

```
            break;
        }
    }

    return IsUTF8;
}

// Convert char to Wchar.
bool Char2Wchar(const char *pStr, wchar_t *pOutWStr, int nOutStrSize)
{
    if (!pStr || !pOutWStr)
    {
        return false;
    }

    bool bIsUTF = IsStrUTF8(pStr, strlen(pStr));
    UINT nChgType = bIsUTF ? CP_UTF8 : CP_ACP;

    int iLen = MultiByteToWideChar(nChgType, 0, (LPCSTR)pStr, -1, NULL, 0);

    memset(pOutWStr, 0, sizeof(wchar_t) * nOutStrSize);

    if (iLen >= nOutStrSize)
    {
        iLen = nOutStrSize - 1;
    }

    MultiByteToWideChar(nChgType, 0, (LPCSTR)pStr, -1, pOutWStr, iLen);

    pOutWStr[iLen] = 0;

    return true;
}

// Convert Wchar to char.
bool Wchar2char(wchar_t *pOutWStr, char *pStr)
{
    if (!pStr || !pOutWStr)
    {
        return false;
    }

    int nLen =  WideCharToMultiByte(CP_ACP, 0, pOutWStr, wcslen(pOutWStr), NULL, 0, NULL, NULL);

    WideCharToMultiByte(CP_ACP, 0 , pOutWStr, wcslen(pOutWStr), pStr, nLen, NULL, NULL);

    pStr[nLen] = '\0';

    return true;
}

// Print the device information.
```

```
bool PrintDeviceInfo(MV_CODEREADER_DEVICE_INFO* pstMVDevInfo)
{
    if (NULL == pstMVDevInfo)
    {
        printf("The Pointer of pstMVDevInfo is NULL!\n");
        return false;
    }
    if (pstMVDevInfo->nTLayerType == MV_CODEREADER_GIGE_DEVICE)
    {
        int nIp1 = ((pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0xff000000) >> 24);
        int nIp2 = ((pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0x00ff0000) >> 16);
        int nIp3 = ((pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0x0000ff00) >> 8);
        int nIp4 = (pstMVDevInfo->SpecialInfo.stGigEInfo.nCurrentIp & 0x000000ff);

        // Print current IP and the user defined name.
        printf("CurrentIp: %d.%d.%d.%d\n" , nIp1, nIp2, nIp3, nIp4);
        wchar_t strWchar[16] = {0};
        Char2Wchar((char*)pstMVDevInfo->SpecialInfo.stGigEInfo.chUserDefinedName, strWchar, 16);
        Wchar2char(strWchar, (char*)pstMVDevInfo->SpecialInfo.stGigEInfo.chUserDefinedName);
        printf("UserDefinedName: %s\n\n" , pstMVDevInfo->SpecialInfo.stGigEInfo.chUserDefinedName);
    }
    else if (pstMVDevInfo->nTLayerType == MV_CODEREADER_USB_DEVICE)
    {
        wchar_t strWchar[16] = {0};
        Char2Wchar((char*)pstMVDevInfo->SpecialInfo.stUsb3VInfo.chUserDefinedName, strWchar, 16);
        Wchar2char(strWchar, (char*)pstMVDevInfo->SpecialInfo.stUsb3VInfo.chUserDefinedName);
        printf("UserDefinedName: %s\n\n", pstMVDevInfo->SpecialInfo.stUsb3VInfo.chUserDefinedName);
    }
    else
    {
        printf("Not support.\n");
    }

    return true;
}

void SaveImageFile(MV_CODEREADER_SAVE_IMAGE_PARAM_EX* pstSaveParam)
{
    if (NULL == pstSaveParam && NULL == pstSaveParam->pImageBuffer)
    {
        return;
    }

    char chFile[128] = {0};
    sprintf_s(chFile, 128, "Image_h%d_w%d_%d.bmp", pstSaveParam->nHeight, pstSaveParam->nWidth, g_nSaveNum);
    FILE* pFile = NULL;
    fopen_s(&pFile, chFile, "wb");
    if (NULL == pFile)
    {
        printf("fopen_s failed, chFile[%s]\n", chFile);
        return;
    }
```

```
    fwrite(pstSaveParam->pImageBuffer, 1, pstSaveParam->nImageLen, pFile);
    fclose(pFile);
    return;
}

void SaveJpegFile(unsigned char* pFrameData, MV_CODEREADER_IMAGE_OUT_INFO_EX2* pstFrameInfo)
{
    if (NULL == pFrameData && NULL == pstFrameInfo)
    {
        return;
    }

    char chFile[128] = {0};
    sprintf_s(chFile, 128, "Image_h%d_w%d_%d.jpeg", pstFrameInfo->nHeight, pstFrameInfo->nWidth, g_nSaveNum);
    FILE* pFile = NULL;
    fopen_s(&pFile, chFile, "wb");
    if (NULL == pFile)
    {
        printf("fopen_s failed, chFile[%s]\n", chFile);
        return;
    }

    fwrite(pFrameData, 1, pstFrameInfo->nFrameLen, pFile);
    fclose(pFile);
}

unsigned int __stdcall SaveImageThread(void* pParam)
{
    if (NULL == pParam)
    {
        return -1;
    }

    void* hDevHandle = pParam;
    unsigned char* pOutImageBuffer = NULL;
    unsigned int   nOutBufferSize = 0;
    int nRet = MV_CODEREADER_OK;
    CODEREADER_IMAGE_OUT_INFO stCoderFrameInfo = {0};
    memset(&stCoderFrameInfo, 0, sizeof(CODEREADER_IMAGE_OUT_INFO));
    MV_CODEREADER_RESULT_BCR_EX2* pstBcrResult = NULL;
    MV_CODEREADER_OCR_INFO_LIST* pstOcrResult = NULL;
    while(g_bRunState)
    {
        CLock lock(g_Lock);
        if (g_lstFrameInfoList.empty())
        {
            Sleep(50);
            continue;
        }
        stCoderFrameInfo = g_lstFrameInfoList.front();
        g_lstFrameInfoList.pop_front();
```

```
    pstBcrResult =
(MV_CODEREADER_RESULT_BCR_EX2*)stCoderFrameInfo.stFrameInfo.UnparsedBcrList.pstCodeListEx2;
    pstOcrResult = (MV_CODEREADER_OCR_INFO_LIST*)stCoderFrameInfo.stFrameInfo.UnparsedOcrList.pstOcrList;

    unsigned int nSize = 0;
    if (stCoderFrameInfo.stFrameInfo.enPixelType == PixelType_CodeReader_Gvsp_Jpeg)
    {
      SaveJpegFile(stCoderFrameInfo.pFrameData, &stCoderFrameInfo.stFrameInfo);
    }
    else
    {
      nSize = stCoderFrameInfo.stFrameInfo.nWidth * stCoderFrameInfo.stFrameInfo.nHeight * 4;
      if (NULL == pOutImageBuffer || nOutBufferSize < nSize)
      {
        if (pOutImageBuffer)
        {
          free(pOutImageBuffer);
          pOutImageBuffer = NULL;
          nOutBufferSize = 0;
        }

        pOutImageBuffer = (unsigned char*)malloc(nSize);
        if (NULL == pOutImageBuffer)
        {
          printf("malloc failed\n");
          break;
        }
        nOutBufferSize = nSize;
      }
      memset(pOutImageBuffer, 0, nOutBufferSize);

      // The image are saved in the buffer in BMP format.
      MV_CODEREADER_SAVE_IMAGE_PARAM_EX stSaveParam = {0};
      memset(&stSaveParam, 0, sizeof(MV_CODEREADER_SAVE_IMAGE_PARAM_EX));
      stSaveParam.pData = stCoderFrameInfo.pFrameData;
      stSaveParam.nDataLen = stCoderFrameInfo.stFrameInfo.nFrameLen;
      stSaveParam.enPixelType = stCoderFrameInfo.stFrameInfo.enPixelType;
      stSaveParam.nWidth = stCoderFrameInfo.stFrameInfo.nWidth;
      stSaveParam.nHeight = stCoderFrameInfo.stFrameInfo.nHeight;

      stSaveParam.pImageBuffer = pOutImageBuffer;
      stSaveParam.nBufferSize = nOutBufferSize;
      stSaveParam.enImageType = MV_CODEREADER_Image_Bmp;

      nRet = MV_CODEREADER_SaveImage(hDevHandle, &stSaveParam);
      if (nRet != MV_CODEREADER_OK)
      {
        if (MV_CODEREADER_E_BUFOVER == nRet)
        {
          if (stSaveParam.pImageBuffer)
          {
```

```
                free(stSaveParam.pImageBuffer);
                stSaveParam.pImageBuffer = NULL;
                stSaveParam.nBufferSize = 0;
            }

            stSaveParam.pImageBuffer = (unsigned char*)malloc(stSaveParam.nImageLen);
            if (NULL == stSaveParam.pImageBuffer)
            {
                printf("malloc failed\n");
                break;
            }
            stSaveParam.nBufferSize = stSaveParam.nImageLen;
            memset(stSaveParam.pImageBuffer, 0, stSaveParam.nBufferSize);

            nRet = MV_CODEREADER_SaveImage(hDevHandle, &stSaveParam);
            if (nRet != MV_CODEREADER_OK)
            {
                printf("Save Image failed, nRet[%#X]\n", nRet);
            }
            else
            {
                SaveImageFile(&stSaveParam);
            }
        }
        else
        {
            printf("Save Image failed, nRet[%#X]\n", nRet);
        }
    }
    else
    {
        SaveImageFile(&stSaveParam);
    }
}

printf("CodeNum[%d]\n", pstBcrResult->nCodeNum);
/*for (int i = 0; i < pstBcrResult->nCodeNum; i++)
{
    wchar_t strWchar[MV_CODEREADER_MAX_BCR_CODE_LEN_EX] = {0};
    Char2Wchar((char*)pstBcrResult->stBcrInfoEx2[i].chCode, strWchar,
MV_CODEREADER_MAX_BCR_CODE_LEN_EX);
    Wchar2char(strWchar, (char*)pstBcrResult->stBcrInfoEx2[i].chCode);
    printf("Get CodeInfo: CodeNum[%d] CodeEx[%s]\n", i, pstBcrResult->stBcrInfoEx2[i].chCode);
}*/

printf("OcrAllNum[%d]\n", pstOcrResult->nOCRAllNum);

/*for (int i = 0; i < pstOcrResult->nOCRAllNum; i ++)
{
    printf("Get OcrInfo: OCRAllNum[%d] rowIndex[%d] chOcr[%s] ocrLen[%d]\r\n",
        pstOcrResult->nOCRAllNum, i,
        pstOcrResult->stOcrRowInfo[i].chOcr,
```

```
            pstOcrResult->stOcrRowInfo[i].nOcrLen);
        }*/

        if (stCoderFrameInfo.pFrameData)
        {
            free(stCoderFrameInfo.pFrameData);
            stCoderFrameInfo.pFrameData = NULL;
        }
        if (pstBcrResult)
        {
            free(pstBcrResult);
            pstBcrResult = NULL;
        }
        if (pstOcrResult)
        {
            free(pstOcrResult);
            pstOcrResult = NULL;
        }
    }

    if (stCoderFrameInfo.pFrameData)
    {
        free(stCoderFrameInfo.pFrameData);
        stCoderFrameInfo.pFrameData = NULL;
    }
    if (pstBcrResult)
    {
        free(pstBcrResult);
        pstBcrResult = NULL;
    }
    if (pstOcrResult)
    {
        free(pstOcrResult);
        pstOcrResult = NULL;
    }

    if (pOutImageBuffer)
    {
        free(pOutImageBuffer);
        pOutImageBuffer = NULL;
        nOutBufferSize = 0;
    }
    return 0;
}

// Register the callback function for image data.
void __stdcall ImageCallBack(unsigned char * pData, MV_CODEREADER_IMAGE_OUT_INFO_EX2* pFrameInfo, void*
pUser)
{
    if (pFrameInfo)
    {
        MV_CODEREADER_RESULT_BCR_EX2* stBcrResult = (MV_CODEREADER_RESULT_BCR_EX2*)pFrameInfo-
```

```
>UnparsedBcrList.pstCodeListEx2;
    MV_CODEREADER_OCR_INFO_LIST* stOcrResult = (MV_CODEREADER_OCR_INFO_LIST*)pFrameInfo-
>UnparsedOcrList.pstOcrList;
    MV_CODEREADER_RESULT_BCR_EX2* pstCodeListEx2 = NULL;
    MV_CODEREADER_OCR_INFO_LIST* pstOcrList = NULL;
    printf("Get One Frame: nChannelID[%d] Width[%d], Height[%d], nFrameNum[%d], nTriggerIndex[%d]\n",
      pFrameInfo->nChannelID, pFrameInfo->nWidth, pFrameInfo->nHeight, pFrameInfo->nFrameNum, pFrameInfo-
>nTriggerIndex);

    if (g_nSaveNum++ < 10)
    {
      CODEREADER_IMAGE_OUT_INFO stCoderFrameInfo = {0};
      memset(&stCoderFrameInfo, 0, sizeof(CODEREADER_IMAGE_OUT_INFO));
      // Allocate the image data buffer.
      if (NULL == stCoderFrameInfo.pFrameData)
      {
        stCoderFrameInfo.pFrameData = (unsigned char*)malloc(pFrameInfo->nFrameLen);
        if (NULL == stCoderFrameInfo.pFrameData)
        {
          printf("[ImageCallBack] malloc failed\n");
          return;
        }
        memset(stCoderFrameInfo.pFrameData, 0, pFrameInfo->nFrameLen);
      }

      memcpy_s(stCoderFrameInfo.pFrameData, pFrameInfo->nFrameLen, pData, pFrameInfo->nFrameLen);

      // Allocate the barcode information buffer.
      if (NULL == pstCodeListEx2)
      {
        pstCodeListEx2 =
(MV_CODEREADER_RESULT_BCR_EX2*)malloc(sizeof(MV_CODEREADER_RESULT_BCR_EX2));
        if (NULL == pstCodeListEx2)
        {
          printf("[ImageCallBack] malloc failed\n");
          return;
        }
      }
      memcpy_s(pstCodeListEx2, sizeof(MV_CODEREADER_RESULT_BCR_EX2), stBcrResult,
sizeof(MV_CODEREADER_RESULT_BCR_EX2));

      // Allocate the OCR information buffer.
      if (NULL == pstOcrList)
      {
        pstOcrList = (MV_CODEREADER_OCR_INFO_LIST*)malloc(sizeof(MV_CODEREADER_OCR_INFO_LIST));
        if (NULL == pstOcrList)
        {
          printf("[ImageCallBack] malloc failed\n");
          return;
        }
      }
      memcpy_s(pstOcrList, sizeof(MV_CODEREADER_OCR_INFO_LIST), pstOcrList,
```

```
sizeof(MV_CODEREADER_OCR_INFO_LIST));

        // Copy frame data and other data.
        memcpy_s(&stCoderFrameInfo.stFrameInfo, sizeof(MV_CODEREADER_IMAGE_OUT_INFO_EX2), pFrameInfo,
sizeof(MV_CODEREADER_IMAGE_OUT_INFO_EX2));
        stCoderFrameInfo.stFrameInfo.UnparsedOcrList.pstOcrList = pstOcrList;
        stCoderFrameInfo.stFrameInfo.UnparsedBcrList.pstCodeListEx2 = pstCodeListEx2;

        CLock lock(g_Lock);
        g_lstFrameInfoList.push_back(stCoderFrameInfo);
      }
    }
}

// Main processing function
int main()
{
    int nRet = MV_CODEREADER_OK;
    void* handle = NULL;
    HANDLE hThread = NULL;
    unsigned int nThreadId = 0;

    do
    {
      // Enumerate devices.
      MV_CODEREADER_DEVICE_INFO_LIST stDeviceList;
      memset(&stDeviceList, 0, sizeof(MV_CODEREADER_DEVICE_INFO_LIST));
      nRet = MV_CODEREADER_EnumDevices(&stDeviceList, MV_CODEREADER_GIGE_DEVICE);
      if (MV_CODEREADER_OK != nRet)
      {
        printf("Enum Devices fail! nRet [0x%x]\n", nRet);
        break;
      }

      if (stDeviceList.nDeviceNum > 0)
      {
        for (unsigned int i = 0; i < stDeviceList.nDeviceNum; i++)
        {
          printf("[device %d]:\n", i);
          MV_CODEREADER_DEVICE_INFO* pDeviceInfo = stDeviceList.pDeviceInfo[i];
          if (NULL == pDeviceInfo)
          {
            break;
          }
          PrintDeviceInfo(pDeviceInfo);
        }
      }
      else
      {
        printf("Find No Devices!\n");
        break;
      }
```

```
printf("Please Intput camera index:");
unsigned int nIndex = 0;
scanf("%d", &nIndex);

if (nIndex >= stDeviceList.nDeviceNum)
{
   printf("Intput error!\n");
   break;
}

// Select the device and create a handle.
nRet = MV_CODEREADER_CreateHandle(&handle, stDeviceList.pDeviceInfo[nIndex]);
if (MV_CODEREADER_OK != nRet)
{
   printf("Create Handle fail! nRet [0x%x]\n", nRet);
   break;
}

// Open the device.
nRet = MV_CODEREADER_OpenDevice(handle);
if (MV_CODEREADER_OK != nRet)
{
   printf("Open Device fail! nRet [0x%x]\n", nRet);
   break;
}

// Set the trigger mode to off.
nRet = MV_CODEREADER_SetEnumValue(handle, "TriggerMode", MV_CODEREADER_TRIGGER_MODE_OFF);
if (MV_CODEREADER_OK != nRet)
{
   printf("Set Trigger Mode fail! nRet [0x%x]\n", nRet);
   break;
}

// Register image callback.
nRet = MV_CODEREADER_RegisterImageCallBackEx2(handle, ImageCallBack, handle);
if (MV_CODEREADER_OK != nRet)
{
   printf("Register Image CallBackEx2 fail! nRet [0x%x]\n", nRet);
   break;
}

g_nSaveNum = 0;
g_bRunState = true;
hThread = (HANDLE)_beginthreadex(NULL, 0, &SaveImageThread, handle, 0, &nThreadId);
if (NULL == hThread)
{
   printf("creat SaveImage Thread failed\n");
   break;
}
```

```
    // Start acquisition (getting stream).
    nRet = MV_CODEREADER_StartGrabbing(handle);
    if (MV_CODEREADER_OK != nRet)
    {
        printf("Start Grabbing fail! nRet [0x%x]\n", nRet);
        break;
    }

    printf("Press a key to stop grabbing.\n");
    WaitForKeyPress();

    // Stop acqusition.
    nRet = MV_CODEREADER_StopGrabbing(handle);
    if (MV_CODEREADER_OK != nRet)
    {
        printf("Stop Grabbing fail! nRet [0x%x]\n", nRet);
        break;
    }

    g_bRunState = false;
    Sleep(100);

    if (hThread)
    {
        DWORD dwRet = WaitForSingleObject(hThread, INFINITE);
        if (dwRet == WAIT_TIMEOUT)
        {
            TerminateThread(hThread, 0);
        }
    }
    CloseHandle(hThread);
  } while (0);

  if (handle != NULL)
  {
    // Close the device.
    // Destroy the handle.
    MV_CODEREADER_CloseDevice(handle);
    MV_CODEREADER_DestroyHandle(handle);
    handle = NULL;
  }

  CLock lock(g_Lock);
  if (!g_lstFrameInfoList.empty())
  {
    CODEREADER_IMAGE_OUT_INFO stCoderFrameInfo = g_lstFrameInfoList.front();
    g_lstFrameInfoList.pop_front();

    MV_CODEREADER_RESULT_BCR_EX2* pstBcrResult =
(MV_CODEREADER_RESULT_BCR_EX2*)stCoderFrameInfo.stFrameInfo.UnparsedBcrList.pstCodeListEx2;
    MV_CODEREADER_OCR_INFO_LIST* pstOcrResult =
(MV_CODEREADER_OCR_INFO_LIST*)stCoderFrameInfo.stFrameInfo.UnparsedOcrList.pstOcrList;
```

```
    if (stCoderFrameInfo.pFrameData)
    {
        free(stCoderFrameInfo.pFrameData);
        stCoderFrameInfo.pFrameData = NULL;
    }
    if (pstBcrResult)
    {
        free(pstBcrResult);
        pstBcrResult = NULL;
    }
    if (pstOcrResult)
    {
        free(pstOcrResult);
        pstOcrResult = NULL;
    }
  }

  if (hThread)
  {
    DWORD dwRet = WaitForSingleObject(hThread, INFINITE);
    if (dwRet == WAIT_TIMEOUT)
    {
        TerminateThread(hThread, 0);
    }
  }
  CloseHandle(hThread);
  printf("Press a key to exit.\n");
  WaitForKeyPress();

  return 0;
}
```

Appendix D. Error Code

You can search for the detailed error descriptions according to returned error codes or error names.

Error Code	Error Name	Description
Success Status Code		
0x00000000	MV_CODEREADER_OK	Succeeded.
General Error (from 0x80020000 to 0x800200FF)		
0x80020000	MV_CODEREADER_E_HANDLE	Error or invalid handle
0x80020001	MV_CODEREADER_E_SUPPORT	The function is not supported
0x80020002	MV_CODEREADER_E_BUFOVER	Buffer is full
0x80020003	MV_CODEREADER_E_CALLORDER	Incorrect call order
0x80020004	MV_CODEREADER_E_PARAMETER	Incorrect parameter
0x80020005	MV_CODEREADER_E_RESOURCE	Resource request failed.
0x80020006	MV_CODEREADER_E_NODATA	No data
0x80020007	MV_CODEREADER_E_PRECONDITION	Incorrect precondition, or running environment changed.
0x80020008	MV_CODEREADER_E_VERSION	Version is mismatched
0x80020009	MV_CODEREADER_E_NOENOUGH_BUF	Insufficient memory
0x8002000A	MV_CODEREADER_E_ABNORMAL_IMAGE	Abnormal image. Incomplete image caused by packet loss.
0x8002000B	MV_CODEREADER_E_LOAD_LIBRARY	Importing DLL failed.
0x8002000C	MV_CODEREADER_E_NOOUTBUF	No output buffer
0x8002000F	MV_CODEREADER_E_FILE_PATH	Incorrect file path
0x800200FF	MV_CODEREADER_E_UNKNOW	Unknown error
GenICam Related Error (from 0x80020100 to 0x800201FF)		
0x80020100	MV_CODEREADER_E_GC_GENERIC	Generic error
0x80020101	MV_CODEREADER_E_GC_ARGUMENT	Invalid parameter
0x80020102	MV_CODEREADER_E_GC_RANGE	The value is out of range.

Error Code	Error Name	Description
0x80020103	MV_CODEREADER_E_GC_PROPERTY	Attribute error
0x80020104	MV_CODEREADER_E_GC_RUNTIME	Running environment error
0x80020105	MV_CODEREADER_E_GC_LOGICAL	Incorrect logic
0x80020106	MV_CODEREADER_E_GC_ACCESS	Node accessing condition error
0x80020107	MV_CODEREADER_E_GC_TIMEOUT	Timed out
0x80020108	MV_CODEREADER_E_GC_DYNAMICCAST	Conversion exception
0x800201FF	MV_CODEREADER_E_GC_UNKNOW	GenICam unknown error
GigE_STATUS Reltaed Errors (from 0x80020200 to 0x800202FF)		
0x80020200	MV_CODEREADER_E_NOT_IMPLEMENTED	Command is not supported by the device.
0x80020201	MV_CODEREADER_E_INVALID_ADDRESS	Target address does not exist.
0x80020202	MV_CODEREADER_E_WRITE_PROTECT	Target address is not writable.
0x80020203	MV_CODEREADER_E_ACCESS_DENIED	No access permission
0x80020204	MV_CODEREADER_E_BUSY	Device is busy, or network is disconnected.
0x80020205	MV_CODEREADER_E_PACKET	Network packet error
0x80020206	MV_CODEREADER_E_NETER	Network error
GigE Cameras Related Error(s)		
0x80020221	MV_CODEREADER_E_IP_CONFLICT	IP address conflicted.
USB_STATUS Related Errors (from 0x80020300 to 0x800203FF)		
0x80020300	MV_CODEREADER_E_USB_READ	USB read error
0x80020301	MV_CODEREADER_E_USB_WRITE	USB write error
0x80020302	MV_CODEREADER_E_USB_DEVICE	Device exception
0x80020303	MV_CODEREADER_E_USB_GENICAM	GenICam error
0x80020304	MV_CODEREADER_E_USB_BANDWIDTH	Insufficient bandwidth
0x80020305	MV_CODEREADER_E_USB_DRIVER	Driver is mismatched, or the driver is not installed.
0x800203FF	MV_CODEREADER_E_USB_UNKNOW	USB unknown error
Upgrade Related Errors (from 0x80020400 to 0x800204FF)		

Error Code	Error Name	Description
0x80020400	MV_CODEREADER_E_UPG_MIN_ERRCODE	Minimum error code of upgrade module
0x80020400	MV_CODEREADER_E_UPG_FILE_MISMATCH	Upgrade firmware is mismatched.
0x80020401	MV_CODEREADER_E_UPG_LANGUSGE_ MISMATCH	Upgrade firmware language is mismatched.
0x80020402	MV_CODEREADER_E_UPG_CONFLICT	Upgrading conflicted (repeated upgrading requests during device upgrade).
0x80020403	MV_CODEREADER_E_UPG_INNER_ERR	Camera internal error during upgrade.
0x80020404	MV_CODEREADER_E_UPG_REGRESH_TYPE_ ERR	Acquiring camera model failed.
0x80020405	MV_CODEREADER_E_UPG_COPY_FPGABIN_ ERR	Copying FPGA file failed.
0x80020406	MV_CODEREADER_E_UPG_ZIPEXTRACT_ERR	Extracting ZIP file failed.
0x80020407	MV_CODEREADER_E_UPG_DAVEXTRACT_ ERR	Extracting DAV file failed.
0x80020408	MV_CODEREADER_E_UPG_DAVCOMPRESS_ ERR	Compressing ZIP file failed.
0x80020409	MV_CODEREADER_E_UPG_ZIPCOMPRESS_ ERR	Compressing DAV file failed.
0x80020410	MV_CODEREADER_E_UPG_GET_PROGRESS_ TIMEOUT_ERR	Upgrade progress acquisition timed out.
0x80020411	MV_CODEREADER_E_UPG_SEND_QUERY_ PROGRESS_ERR	Failed to send progress query instruction.
0x80020412	MV_CODEREADER_E_UPG_RECV_QUERY_ PROGRESS_ERR	Failed to receive progress query instruction.
0x80020413	MV_CODEREADER_E_UPG_GET_QUERY_ PROGRESS_ERR	Getting query progress failed.
0x80020414	MV_CODEREADER_E_UPG_GET_MAX_ QUERY_PROGRESS_ERR	Getting fastest progress failed.
0x80020465	MV_CODEREADER_E_UPG_CHECKT_PACKET_ FAILED	Verifying file failed.

Error Code	Error Name	Description
0x80020466	MV_CODEREADER_E_UPG_FPGA_ PROGRAM_FAILED	FPGA program upgrade failed.
0x80020467	MV_CODEREADER_E_UPG_WATCHDOG_ FAILED	Watchdog upgrade failed.
0x80020468	MV_CODEREADER_E_UPG_CAMERA_AND_ BARE_FAILED	Bare camera upgrade failed.
0x80020469	MV_CODEREADER_E_UPG_RETAIN_CONFIG_ FAILED	Retaining configuration file failed.
0x8002046A	MV_CODEREADER_E_UPG_FPGA_DRIVER_ FAILED	FPGA drive upgrade failed.
0x8002046B	MV_CODEREADER_E_UPG_SPI_DRIVER_ FAILED	SPI drive upgrade failed.
0x8002046C	MV_CODEREADER_E_UPG_REBOOT_ SYSTEM_FAILED	Restarting failed.
0x8002046D	MV_CODEREADER_E_UPG_UPSELF_FAILED	Service upgrade failed.
0x8002046E	MV_CODEREADER_E_UPG_STOP_RELATION_ PROGRAM_FAILED	Stop related service failed.
0x8002046F	MV_CODEREADER_E_UPG_DEVCIE_TYPE_ INCONSISTENT	Inconsistent device type
0x80020470	MV_CODEREADER_E_UPG_READ_ENCRYPT_ INFO_FAILED	Read encryption message failed.
0x80020471	MV_CODEREADER_E_UPG_PLAT_TYPE_ INCONSISTENT	Incorrect device platform
0x80020472	MV_CODEREADER_E_UPG_CAMERA_TYPE_ INCONSISTENT	Incorrect camera model
0x80020473	MV_CODEREADER_E_UPG_DEVICE_ UPGRADING	Incorrect camera model
0x80020474	MV_CODEREADER_E_UPG_UNZIP_FAILED	Upgrade package decompression failed.
0x80020475	MV_CODEREADER_E_UPG_BLE_DISCONNECT	PDA bluetooth is disconnected.
0x80020476	MV_CODEREADER_E_UPG_BATTERY_ NOTENOUGH	Low battery.

Error Code	Error Name	Description
0x80020477	MV_CODEREADER_E_UPG_RTC_NOT_ PRESENT	PDA is not on the base.
0x80020478	MV_CODEREADER_E_UPG_APP_ERR	Failed to upgrade the app.
0x80020479	MV_CODEREADER_E_UPG_L3_ERR	Failed to upgrade L3.
0x8002047A	MV_CODEREADER_E_UPG_MCU_ERR	Failed to upgrade MCU.
0x8002047B	MV_CODEREADER_E_UPG_PLATFORM_ DISMATCH	Platform does not match.
0x8002047C	MV_CODEREADER_E_UPG_TYPE_DISMATCH	Model does not match.
0x8002047D	MV_CODEREADER_E_UPG_SPACE_ DISMATCH	Space does not match.
0x8002047E	MV_CODEREADER_E_UPG_MEM_DISMATCH	Memory does not match.
0x8002047F	MV_CODEREADER_E_UPG_NET_TRANS_ ERROR	Network transmission exception. Please upgrade again.
0x800204FF	MV_CODEREADER_E_UPG_UNKNOW	Upgrade unknown error
Network Components Related Errors (from 0x80020500 to 0x800205FF)		
0x80020500	MV_CODEREADER_E_CREAT_SOCKET	Creating socket error
0x80020501	MV_CODEREADER_E_BIND_SOCKET	Binding error
0x80020502	MV_CODEREADER_E_CONNECT_SOCKET	Connection error
0x80020503	MV_CODEREADER_E_GET_HOSTNAME	Host name getting error
0x80020504	MV_CODEREADER_E_NET_WRITE	Data write error
0x80020505	MV_CODEREADER_E_NET_READ	Data read error
0x80020506	MV_CODEREADER_E_NET_SELECT	Selection error
0x80020507	MV_CODEREADER_E_NET_TIMEOUT	Timed out
0x80020508	MV_CODEREADER_E_NET_ACCEPT	Receive error
0x800205FF	MV_CODEREADER_E_NET_UNKNOW	Network unknown error